

队伍编号	MCB2400002
赛道	B

基于电商品类货量预测的品类分仓多目标规划研究

摘要

随着电子商务的飞速发展，仓储资源配置与成本优化成为企业供应链管理的关键。本文聚焦于多仓库网络中 350 种商品的库存与销量数据，构建了一套货量预测与分仓优化模型，旨在优化仓储成本与资源利用率。

首先，在问题一中，基于对未来三个月仓储需求的预测，本文通过**移动平均法**处理异常数据，利用**随机森林**填补缺失值，确保了数据的完整性与准确性。在此基础上，进行了三种探索性分析，包括品类销量和库存量的**时间趋势**分析、**仓库信息折线图**分析和**品类相关信息三维图**分析，以揭示需求模式和资源使用情况。然后针对销量的时间序列特征，进一步进行了**K-means 聚类**分析，为下一步的预测提供了进一步的信息支持。在预测阶段，本文分别采用**指数平滑**和**ARIMA**模型对品类库存量进行了预测，另通过**XGBoost**、**LightGBM**和**LSTM**模型对品类销售量进行预测，并集成了多种模型的预测结果，为“一品一仓”与“一品多仓”的分仓方案提供了精确的需求数据支持。

在问题二的“一品一仓”情境下，本文构建了多目标优化模型，以**最小化仓储成本**、**最大化品类关联度**、**仓容利用率**和**产能利用率**为优化目标。在多目标优化的算法选择上，采用了改进的**NSGA-III**算法。为解决其高维问题中的收敛与多样性问题，引入了**反向学习**和**动态交叉变异机制**，前者通过扩展初始种群探索未见解空间，提升了解的广度，后者在算法不同进化阶段自适应地调整交叉与变异概率，进一步提升了全局搜索能力和收敛速度。最终，模型优化结果将日仓储成本收敛至**169631 元**，使用**47 个仓库**，并显著提升了品类关联度，为合理分配仓储资源提供了高效方案。

在问题三中，放宽“一品一仓”限制，允许品类分布在不超过 3 个仓库。基于问题二的模型框架，本文进一步优化了目标函数和约束条件，强调通过**关联度的最大化**将同属性品类集中于同一仓库，以减少管理复杂性和履约成本。为此模型新增了多仓库库存与出库分配控制的决策变量，引入了品类的仓库分布比例一致性约束，确保灵活性与分仓一致性。算法方面，引入了**偏好评价**机制，在不同阶段对优化目标进行**差异化加权**，通过**角度惩罚机制**确保解在多目标空间内的均衡性，综合评价函数则进一步增强了解的多样性与优先目标导向。最终优化结果日成本降低至**146558 元**，仓库数量减少至**40 个**，且品类关联度从**974381**提升至**1528489**，表现出仓储成本节约和资源利用率提升的双重效果。

最后，为验证模型的稳健性，本文在模型分析与检验部分设计了**三种情境测试**，模拟了**销量增加 5%**、**仓库容量增加 5%**及**基准情境**下的成本和品类关联度变化，充分展示了模型在不同条件下的适应性与稳定性。

关键词：电商品类分仓 时间序列预测 NSGA-III 偏好评价

目录

摘 要	1
一、问题重述	1
1.1 问题背景	1
1.2 基本问题	1
二、模型假设	2
三、符号说明	2
四、问题一的建模与求解	2
4.1 问题分析	3
4.2 数据预处理	3
4.2.1 异常数据处理	3
4.2.2 缺失数据处理	4
4.3 各品类及仓库数据探索性分析	5
4.3.1 基于关联度的各品类销量及库存量可视化分析	6
4.3.2 仓库数据可视化分析	8
4.3.3 不同品类相关信息可视化分析	9
4.4 各品类销售量序列聚类分析	9
4.4.1 特征因子的构建	9
4.4.2 K-means 聚类算法的建立	10
4.4.3 各品类销售量序列聚类结果分析	11
4.5 各品类库存量预测分析	12
4.5.1 指数平滑预测模型的建立	12
4.5.2 ARIMA 预测模型的建立	13
4.5.3 各品类销售量预测效果评价	13
4.5.4 各品类库存量预测结果分析	14
4.6 各品类销售量预测分析	15
4.6.1 XGBoost 预测模型的构建	15
4.6.2 LightGBM 预测模型的构建	16
4.6.3 LSTM 预测模型的构建	16
4.6.4 各品类销售量预测效果评价	17
4.6.5 各品类销售量预测结果分析	19
五、问题二的建模与求解	19
5.1 问题分析	20
5.2 “一品一仓”品类分仓规划模型的建立	21
5.2.1 决策变量的定义	21
5.2.2 约束条件的设定	21
5.2.3 目标函数的设定	22
5.3 改进的 NSGA-III 算法的应用	23
5.4 “一品一仓”品类分仓规划模型的求解	27
5.4.1 算法迭代结果分析	27
5.4.2 “一品一仓”品类分仓结果分析	28
5.4.3 “一品一仓”品类分仓业务指标分析	28
六、问题三的建模与求解	29
6.1 问题分析	29
6.2 “一品多仓”品类分仓规划模型的建立	30

6.2.1 新增决策变量的定义	30
6.2.2 约束条件的增设及改进	30
6.2.3 目标函数的增设与改进	32
6.3 引入决策者偏好评价的改进 NSGA-III 算法	33
6.4 “一品多仓”品类分仓规划模型的求解	36
6.4.1 算法迭代结果分析	36
6.4.2 “一品多仓”品类分仓结果分析	37
6.4.3 “一品多仓”品类分仓业务指标分析	37
6.4.4 同高级品类、同伴型存放结果分析	38
七、模型的分析与检验	39
7.1 基于成本的鲁棒性分析	39
7.2 基于品类关联度的鲁棒性分析	40
八、模型的评价与推广	40
8.1 模型的优点	41
8.2 模型的缺点	41
8.3 模型的推广	41
参考文献	42
附录	43
附录 1 移动平均处理异常值的程序	44
附录 2 随机森林填补缺失值的程序	45
附录 3 指数平滑、ARIMA 预测库存量的程序	47
附录 4 XGBoost、LightGBM、LSTM 预测销售量的程序	50
附录 5 改进的 NSGA-III 算法的程序	57
附录 6 引入偏好评价的改进的 NSGA-III 算法的程序	61

一、问题重述

本节旨在提取题目中的关键信息，全面概括在多仓库网络中电商品类货量预测及分仓规划的问题背景，并明确求解在未来三个月内，对电商仓储网络的 350 种品类货量进行预测与合理分仓的最优方案的要求，从而更清晰地把握问题的核心要点。

1.1 问题背景

随着电商行业的高速发展和线上购物需求的不断增长，电商仓储物流的精细化管理面临着新的挑战。为了提升仓储管理效率、降低仓储成本，电商企业通常依赖于多个仓库组成的仓群进行商品存储。仓库中的商品根据品类和特性进行分类和打标，以便于库存管理和快速出库。合理的仓储分配方案不仅能优化仓储空间的利用率，还能够显著减少冗余资源投入，提高资源配置效率，从而降低企业的运营成本。

在该电商仓储场景中，准确的货量预测是实现高效分仓规划的关键。通过准确的库存量和销量预测，可以前瞻性地制定仓储资源分配计划，有效降低因空间不足或产能超载而引发的履约风险。具体而言，库存量决定每个仓库所需的存储空间，而销量则直接影响出库的效率和产能需求。因此，为了实现最佳的仓储资源配置，仓储分配方案需综合考虑仓库容量和出库能力的限制。

此外，品类间的关联性也是优化仓储分布的重要因素。将关联度高的品类集中存放在同一仓库，可以减少订单分拣和包装的次数，降低履约成本，提高订单履行的效率。与此同时，电商企业需要在仓库租金、管理成本等方面进行权衡，以实现仓储成本和管理效益的平衡。

因此，针对复杂的仓储条件和运营限制，本次任务需要构建科学的预测和优化模型，基于历史数据和业务约束条件，制定未来三个月的品类货量预测和分仓方案。通过对仓储网络的优化配置，进一步提升仓储管理效率和成本效益，为电商供应链的高效运作提供数据支持和科学依据。

1.2 基本问题

附件提供了电商仓储网络中 350 种商品品类的历史库存量、销量、仓库容量、产能、仓租成本以及品类之间的关联度等数据。为优化仓储管理、提高存储与出库效率，并降低物流运营成本，现需结合实际数据和附件信息，建立科学的预测与分仓规划模型，分析并解决以下问题：

问题一：依据附件 1 和附件 2 的历史库存量和销量数据，假设未来库存量和销量的变化趋势与过去保持一致，建立货量预测模型，对 350 种商品品类在未来 3 个月（7-9 月）的月度库存量和每日销量进行预测。月度库存量需预测月均库存量并填写在表 1 及 result 文件的“月库存预测结果”表单中；每日销量需提供日度预测并填写在表 2 及 result 文件的“日销量预测结果”表单中。

问题二：在问题一预测结果的基础上，假设每种品类只能存放于一个仓库中（即“一品一仓”），并在考虑仓库容量和产能限制的情况下，制定一品一仓的分仓方案。目标是确定应使用的仓库及其对应存放的品类，以尽可能优化仓容与产能的利用效率，并减少仓租成本。最终分仓方案填写在表 3 及 result 文件的“一品一仓分仓方案”表单中。

问题三：在问题二的基础上，放开“一品一仓”的限制，允许品类分布在不超过 3 个仓库中（即“一品多仓”），同时尽量将同伴型或高级品类存放在同一仓库。目标是在满足仓容和产能限制的前提下，通过最大化品类关联度来提高履约效率并降低成本。最终的分仓方案填写在表 4 及 result 文件的“一品多仓分仓方案”表单中，并与问题二的方案进行比较分析。

二、模型假设

- 本文提出以下合理假设：
- 1. 假设不考虑突发情况（如市场剧烈波动）对电商品类库存和销量的影响。
 - 2. 假设未来三个月内各仓库的容量和产能保持不变，不受外部因素影响。
 - 3. 假设品类之间的关联度在预测期内保持稳定。

三、符号说明

本节将文中使用到的相关数据符号进行说明。

符号	含义	单位	备注
mean	品类销售量的平均值	件	变量
std	品类销售量的标准差	件	变量
x_{ij}	品类存放决策	-	二元决策变量
s_i	品类 <i>i</i> 的三个月库存总量	件	变量
d_{it}	品类 <i>i</i> 在日期 <i>t</i> 的日销量	件	变量
P_j	仓库 <i>j</i> 的日出库产能上限	件	常量
c_j	仓库 <i>j</i> 的日租成本	元	常量
a_{ik}	品类 <i>i</i> 和品类 <i>k</i> 的关联度	-	变量
V_j	仓库 <i>j</i> 的容量	件	常量
P_{ci}	交叉概率	-	变量
P_{mi}	变异概率	-	变量
z_j^{min}	理想点	-	变量
$ASF(x, w)$	目标偏量	-	变量
y_{ij}	品类库存分仓决策	-	决策变量
d_{ijt}	品类日销售分仓决策	-	决策变量
r_{ij}	品类分配比例	-	决策变量
h_{ik}	品类 <i>i</i> 和品类 <i>k</i> 是否具有相同属性	-	二元变量
$\lambda_m(t)$	动态加权因子	-	变量
$\vec{u}_i$	解的方向向量	-	变量
θ_i	解与偏好方向的夹角 θ_i	-	变量
P_i	角度惩罚项	-	变量

四、问题一的建模与求解

本节问题一围绕电商仓储网络中的货量预测展开，旨在针对未来 3 个月的仓储需求，预测 350 个品类的库存量与销量。为了实现高效的仓储管理和资源配置，本文从库存量的月度预测及销量的日度预测两个维度入手，构建了一套基于时间

序列的预测模型。具体而言，通过结合历史数据与时间序列趋势特征，采用指数平滑与 ARIMA 等多模型集成的方法，对库存量进行预测；并利用 XGBoost、LightGBM 及 LSTM 模型对日销量序列进行细粒度的预判，以期实现未来库存管理的科学化、精细化。预测结果不仅为后续“一品一仓”分仓方案提供了依据，还为仓储网络的供应链优化决策奠定了数据基础。

4.1 问题分析

问题一的核心在于为电商仓储网络中的 350 种品类，提供未来三个月的库存量和销量预测，以实现仓储资源的合理调配和库存优化管理。该预测问题包括两方面：一是对月度库存量的预测，二是对日销量的预测。首先，库存量预测是通过历史月度数据捕捉仓储需求趋势，以便在未来的每个月保障仓容使用的合理性；其次，销量预测旨在通过每日需求波动的精细化分析，为仓储和出库安排提供更可靠的参考。

为此，本文将问题一建模为一个多步时间序列预测问题，通过对附件中的历史库存量和销量数据进行清洗和分析，以提高预测结果的准确性和可解释性。具体而言，库存量和销量的时间序列具有季节性和趋势性特征，且不同品类的需求波动较大。为解决该问题，本文通过指数平滑和 ARIMA 等经典统计模型来预测库存量，并通过 XGBoost、LightGBM 和 LSTM 等机器学习模型对销量进行预测。多种模型的集成使预测结果能够同时反映整体趋势和细微波动，为仓储和库存管理提供更为可靠的量化依据。

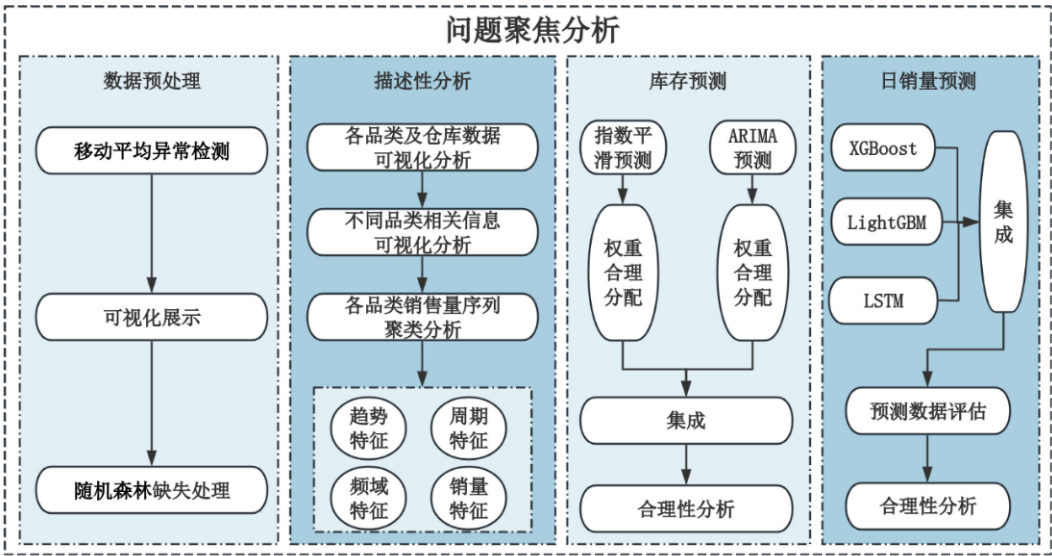


图 1 问题一解决方案整体框架

4.2 数据预处理

数据预处理是进行后续数据分析、数据挖掘的基础步骤，需要替换异常数据，并将数据处理成不含空缺值的形式。

4.2.1 异常数据处理

通过对附件 1（各品类的历史库存量数据）和附件 2（各品类的历史销量数据）进行可视化分析，发现附件 1 没有异常值，而附件 2 部分数据点与总体均值

偏差较大，存在显著的异常值。为确保数据的准确性，本文采用基于 Z-score 的 3σ 原则识别并处理异常值，具体方法包括计算数据的均值和标准差，判断 Z-score 是否大于 3 来识别异常值，再使用移动窗口大小为 5 的均值对异常值进行替换，并对缺失数据点跳过处理。通过重复该过程，逐步剔除异常值，从而减少异常数据对结果的影响。具体步骤如下：

Step 1 计算每种品类历史销量数据的均值和标准差，以便识别异常值。设历史销量数据为 $\{x_1, x_2, \dots, x_n\}$ ，其均值为 $\bar{x}$ ，标准差为 σ ，标准差的计算公式如下：

$$\sigma = \sqrt{\frac{(x_1 - \bar{x})^2 + (x_2 - \bar{x})^2 + \dots + (x_n - \bar{x})^2}{n - 1}} \quad (1)$$

Step 2 对于每个数据点 x_i ，计算其 Z-Score，即数据点与均值之差的绝对值与标准差的比值。公式如下：

$$Z_i = \frac{|x_i - \bar{x}|}{\sigma} \quad (2)$$

当 Z-score 大于 3 时，将该数据点识别为异常值。

Step 3 使用移动窗口大小为 5 的平均值对检测出的异常值进行替换。具体方法是，利用该数据点前后相邻点构成的窗口的均值来替换异常值。例如，若数据点 x_i 被识别为异常值，则用窗口 $\{x_{i-2}, x_{i-1}, x_{i+1}, x_{i+2}\}$ 的均值来替代

$$x_i = \frac{x_{i-2} + x_{i-1} + x_{i+1} + x_{i+2}}{4} \quad (3)$$

Step 4 重复执行步骤 1 到步骤 3，直到不再检测到新的异常值为止。

4.2.2 缺失数据处理

通过对数据进行可视化分析后发现附件 1 的数据完整，没有缺失值，而附件 2 的销售量数据中存在较多缺失情况。针对附件 2 的缺失数据，我们进行了分类处理。下表 X 是针对附件 2 年度数据缺失品类的汇总情况。

表 1 年度数据缺失品类汇总

品类编号	缺失 2022 年 7、8、9 月	缺失 2023 年 4、5、6 月
348	✓	✓
7		✓
332	✓	✓
329	✓	✓
288		✓
345	✓	✓
336	✓	✓
249	✓	✓
339	✓	✓
213	✓	
304		✓
31		✓
134		✓
159	✓	

首先，对上述 2022 年和 2023 年数据全部缺失的品类，统一将缺失值填补为 0。随后，对剩余缺失数据进行可视化分析，发现缺失情况可分为三类：个别值

缺失、连续值缺失和长段值缺失，如下图所示。

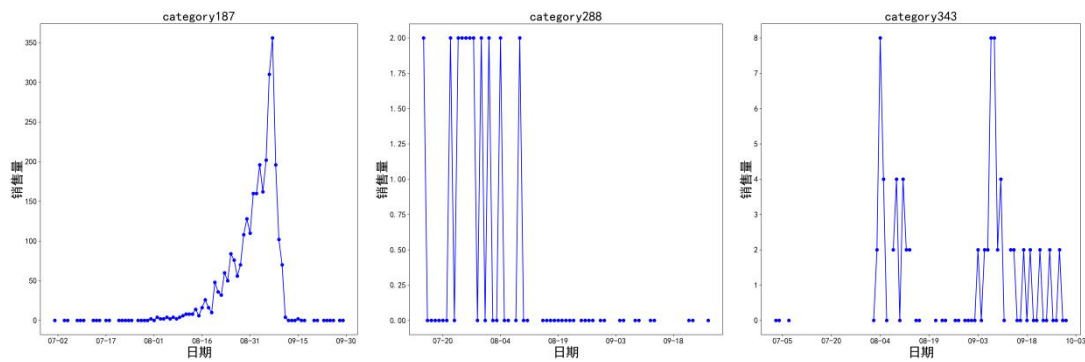


图 2 个别值缺失（左）、连续值缺失（中）、长段值缺失（右）

针对这些多样化的缺失情况，我们采用了随机森林模型进行缺失值填补。随机森林具有良好的稳健性和处理非线性数据的能力，能够综合多个决策树的结果，提供较为准确的预测。通过随机森林算法，我们可以根据数据中的其他特征值，合理推测出缺失数据，从而有效提高数据的完整性和后续分析的准确性。

随机森林是一种集成学习方法，其原理是通过构建多个决策树来进行预测。具体而言，随机森林在训练过程中会从原始数据集中随机抽取多个子样本，并在每个子样本上构建一棵决策树。每棵树在进行节点分裂时，仅随机选择部分特征来寻找最佳分割，从而增加模型的多样性和泛化能力。在预测阶段，随机森林将所有树的预测结果进行投票或平均，从而得出最终的预测值。这种方法尤其适合用于处理缺失数据的填补。通过随机森林模型拟合填补后的序列如下图所示。

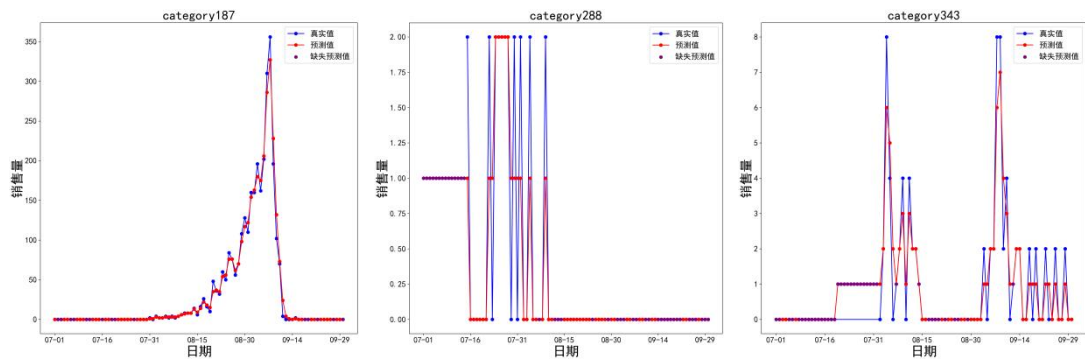


图 3 随机森林模型填补情况

图中展示了随机森林模型对销售量数据中三种缺失情况的填补效果。可以看出，随机森林模型在个别值缺失的情境下，预测值与真实值几乎完全重合，填补效果良好；在连续值缺失的情况下，模型能够捕捉到销售量的波动趋势，填补结果较为平滑；在长段值缺失的情境中，模型填补出的值较好地反映了销售量的主要变化趋势，尽管在某些峰值处存在一定偏差。总体而言，随机森林模型在不同缺失类型下均能较为准确地恢复缺失数据的趋势和特征。

4.3 各品类及仓库数据探索性分析

在本节中，我们对各品类和仓库的数据进行了初步的探索性分析，旨在揭示各品类之间的相似性和关联性，以及仓库资源的分配和利用情况。通过可视化展示和分析，我们试图进一步理解品类销量和库存量在时间序列上的波动规律，特别是对于具有高关联度的品类组合，分析它们在市场需求和库存管理上的相似性或差异性。此外，我们还分析了各仓库的容量、产能和成本结构，以探讨仓储资

源的分布特点及其对仓储决策的影响。通过这些数据的探索性分析，我们为后续优化策略的制定和供应链管理的提升提供了数据支撑和决策依据。

4.3.1 基于关联度的各品类销量及库存量可视化分析

本节对部分品类的销售量和库存量序列进行了可视化展示。我们根据附件 4 中的品类关联度数据，选取了几组关联度较高的品类组合，并绘制了它们的时间序列分布图。通过对这些高关联度品类的销售量和库存量的时间变化进行观察，我们可以更直观地分析它们在不同时间点的变化规律，，为后续的库存优化和供应链决策提供数据支持。

(1) 部分品类销量可视化分析

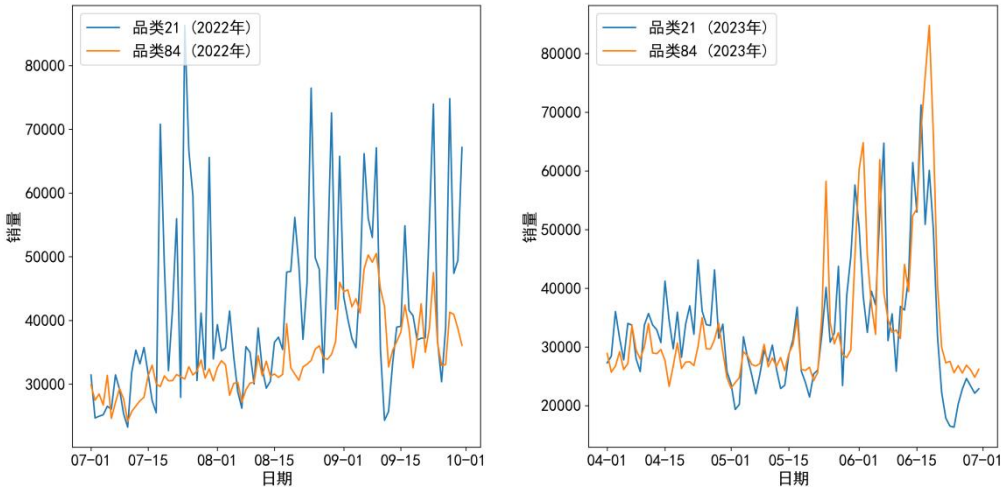


图 4 品类 21 和品类 24 的销售量可视化

从图中可以看出，品类 21 与品类 84 在 2022 年和 2023 年的销售量变化趋势上有很高的相似性，尤其是在特定时间段内呈现出较为一致的波动模式。两品类的销售量在 2022 年 7 月至 10 月和 2023 年 4 月至 7 月的期间均出现明显的波峰和波谷，表明它们在市场需求上可能受到类似因素的影响。此外，2023 年两品类的销售高峰出现时间较为接近，暗示它们的销售量在季节性或节假日等因素驱动下具有同步增长的特征。

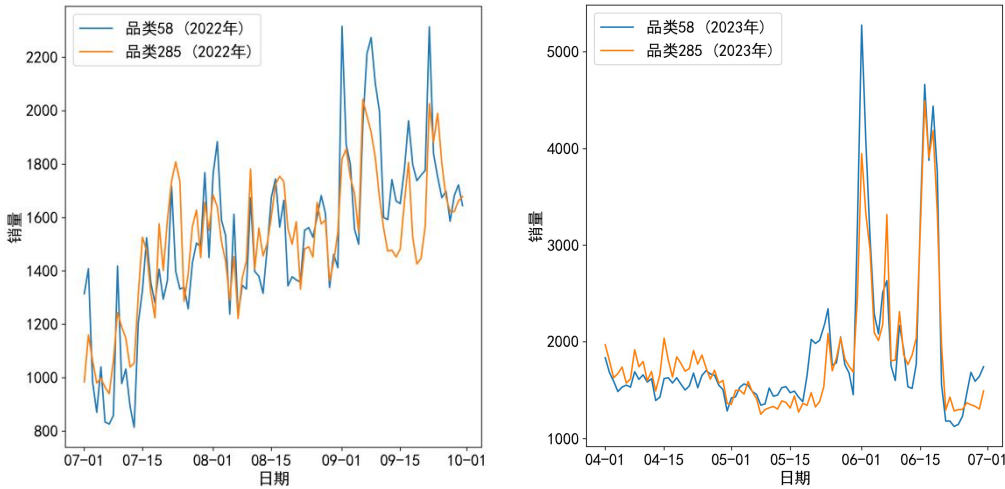


图 5 品类 58 和品类 285 的销售量可视化

从图中可以看到，品类 58 和品类 285 在 2022 年和 2023 年的销售量走势中展现了高度的同步性，尤其是在需求波动的高峰期，二者的销售变化趋势基本一

致。在 2022 年 7 月至 10 月以及 2023 年 5 月至 6 月期间，这两类产品的销量都出现了明显的高峰，并且在相近的时间内达到峰值，表明它们的需求可能受到相似的市场驱动因素影响。

综上所述可以看出，通过对这两组高关联度品类的销售量进行时间序列分析，我们可以观察到这些品类在需求变化上表现出显著的相似性和同步性。这种规律性表明这些高关联度品类可能受到类似的市场因素或季节性需求的影响，因而在相近的时间内经历了相似的销售波动。这一分析为后续的销量预测和库存优化的制定提供了数据支持。

(2) 部分品类库存量可视化分析

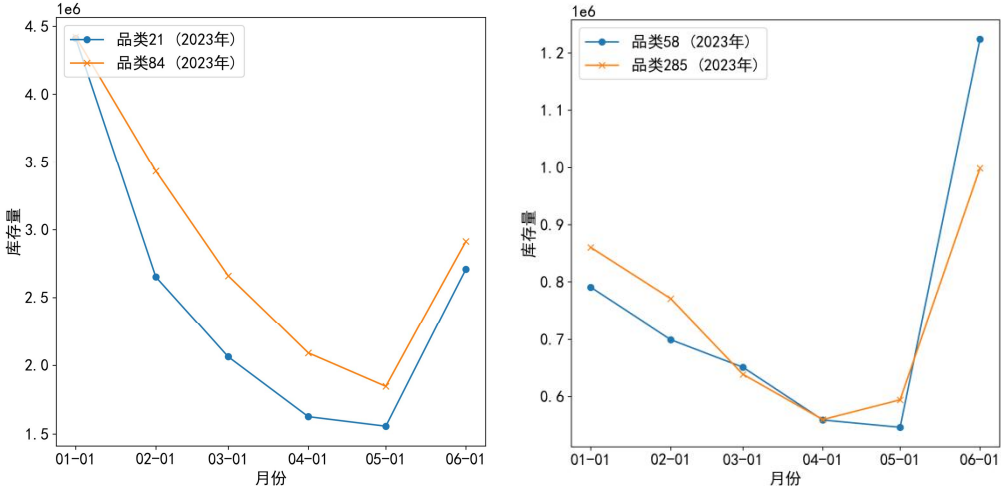


图 6 品类 21 和品类 84、品类 58 和品类 285 的库存量可视化

上面的图展示了 2023 年两组高关联度品类的库存量变化趋势。可以看出，在整个时间周期内，这些品类的库存量变化呈现出显著的同向波动特征。品类 21 和品类 84 在 1 月初库存量均处于较高水平，随后从 1 月到 4 月逐步下降，并在 4 月底达到最低点，之后又逐渐回升，至 6 月时恢复至较高库存量。同样，品类 58 和品类 285 的库存量也表现出类似的趋势。这种同向变化的趋势表明，高关联度品类在库存需求方面存在同步性，可能受到相似的市场需求或季节性因素的影响。

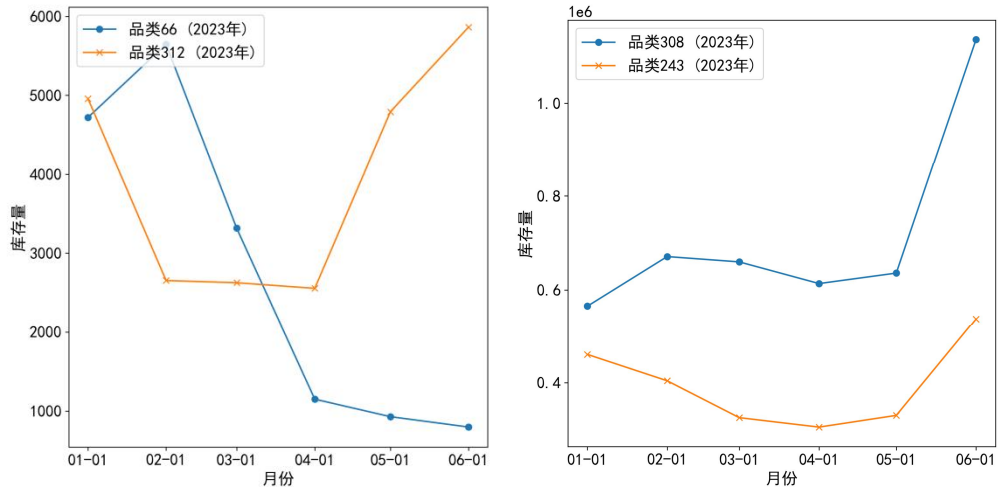


图 7 品类 66 和品类 312、品类 308 和品类 243 的库存量可视化

以上图表展示了 2023 年另外两组高关联度品类的库存量变化情况，但与之前观察到的品类库存量同向变化不同，这些品类在时间序列上的库存变化趋势存

在一定的差异。以品类 66 和品类 312 为例，品类 66 的库存量自年初快速下降，4 月达到最低点后保持低位，而品类 312 的库存量则在年初较高水平后有所下降，但 4 月后库存量再次显著上升。同样，品类 308 和品类 243 的库存趋势也有所不同，在 2 月至 4 月期间，两条曲线出现逐渐拉开，差距越来越大的情况。

综上所述，在高关联度品类的库存管理中，我们观察到既有库存量趋势相近的品类对，也存在库存量趋势相反的品类组合。这种现象反映了不同高关联度品类在实际库存管理和需求规律上的复杂性，而其背后可能涉及到供应链运作、市场需求波动以及产品特性的多方面因素。库存趋势相近的品类可能共享相似的市场需求模式或受到共同的供应链因素影响，而库存趋势相反的品类则可能受到互补性需求、替代性消费或供应链干扰的影响。

因此，高关联度品类并不总是表现出一致的库存趋势，这提示我们在仓储优化中需要细化管理策略。对于需求波动同步的品类，可以采用集中式的管理方法，而对于库存趋势相反的品类组合，则需更加动态地调整仓储资源，以应对需求和供应的不确定性。这种基于库存变化规律的差异化策略，将有助于提高仓储效率，减少库存成本，并增强供应链的适应性和弹性。

4.3.2 仓库数据可视化分析

本小节通过双 Y 轴图对各仓库的容量、产能及成本进行了可视化分析。左侧 Y 轴显示仓库的容量和产能情况，右侧 Y 轴展示各仓库的成本，图中可以直观地观察到容量、产能与成本在不同仓库间的分布关系。通过这种双 Y 轴可视化方式，我们可以更加清晰地理解各仓库的资源配置与成本结构之间的关系，为优化仓储布局提供数据支持。

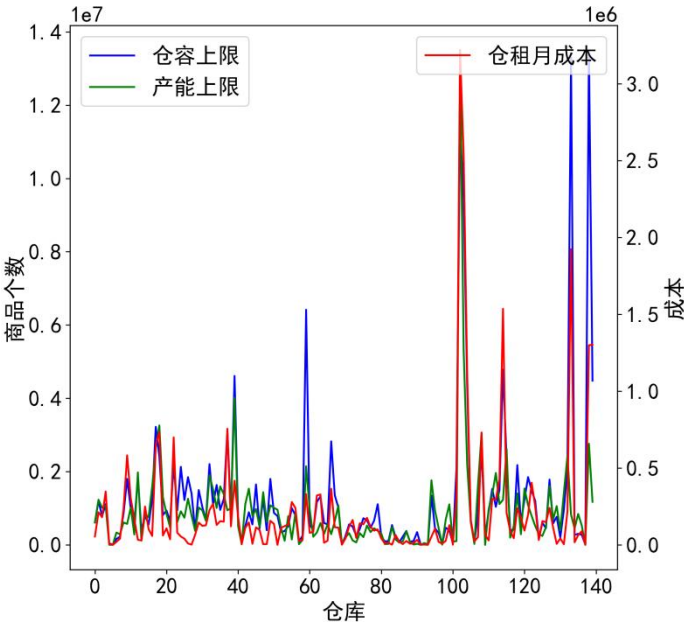


图 8 仓库仓容上限、产能上限及成本折线图

从双 Y 轴图的分析中可以看到，总体上仓库的容量、产能和成本之间存在正相关关系。容量较大的仓库通常具有更高的产能支持能力，同时也伴随着更高的运营成本。这种正相关关系说明，仓库在提升存储和出库能力的同时，也需要承担更高的租赁或运营费用。

具体来看，某些仓库既拥有较大的容量和较高的产能，也显示出较高的成本，适合存放需求量大且需要频繁出库的品类。而对于容量大但产能相对较低的仓库，

由于运营成本相对较低，适合存放低流动性的商品。这些仓库的特点表明它们在成本控制上具备优势，但在出库响应速度方面可能有所限制。因此，在制定仓储策略时，可以优先考虑将高需求、高流动性的品类分配到高产能、高成本的仓库中，而低需求、低流动性的品类则适合分配到成本较低的仓库中，以实现成本和效率的平衡。

4.3.3 不同品类相关信息可视化分析

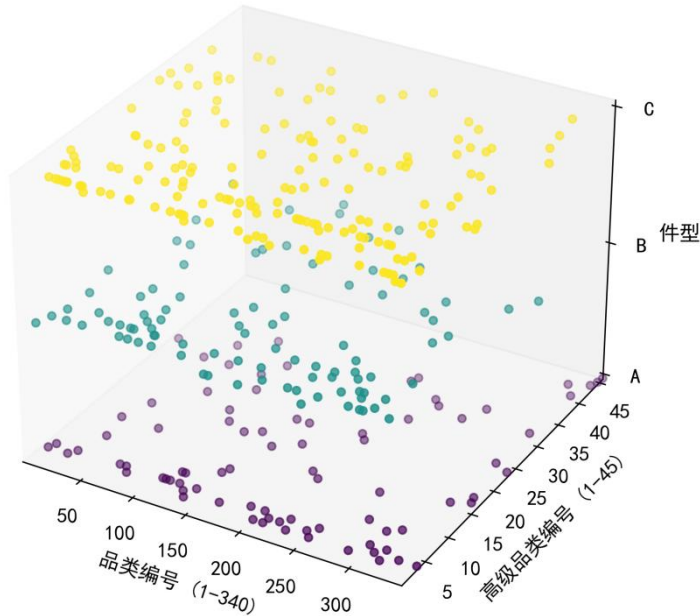


图9 品类信息三维可视化图

这张图展示了不同品类在件型与高级品类编号维度上的分布情况，品类编号从1到340，件型分为A、B、C三种类别。可以观察到，品类在不同件型上的分布存在明显的层次性，品类编号与高级品类编号较高的点主要集中在件型C区域，而件型A区域内的品类数量相对较少且编号集中在较低的范围。

从件型和高级品类编号的分布来看，件型A、B、C之间呈现出一定的层级关系。件型C中的品类数量明显多于其他件型，且分布较为广泛，说明该件型可能对应更多样化的产品。相比之下，件型A的点分布较为稀疏，主要集中在低编号的高级品类上，这可能表明件型A的产品在高级品类上集中于较少的种类，品类扩展较少。通过这种三维可视化，我们可以直观了解不同品类在件型和高级品类上的分布特征，为后续的库存和管理策略提供支持。

4.4 各品类销售量序列聚类分析

本小节针对附件2中各品类的销售量序列数据时间分布进行分析，发现数据涵盖了2022年7月至9月及2023年4月至6月两个时间段。为更好地辅助预测2023年7月至9月的销售量，我们将数据拆分为2022年同期的对比序列和2023年前期的销售序列，分别进行聚类分析。通过比较这两个时间段的聚类结果，探讨它们的相似性和差异性，以揭示销售规律并为后续预测提供支持。

4.4.1 特征因子的构建

本文选取了数据处理后的销售量数据作为研究对象，构建销售量特征、趋势特征、频域特征、周期特征等。具体特征定义如下：

- (1) 销售量特征：mean、std

mean 表示每个品类销售量的平均水平，反映整体的库存和销量水平。std 表示每个品类销售量的波动性，反映库存和销量的稳定程度。

(2) 趋势特征: trend_slope

trend_slope 用来捕捉每个品类销售量数据中的趋势特征，表示销售量随时间的变化方向及速率。趋势斜率的计算方法是通过对时间序列的总趋势。其计算公式为：

$$\text{trend_slope}_i = \beta_{0i} + \beta_{1i} \times n \quad (4)$$

(3) 频域特征: fft_coeff_1

fft_coeff_1 利用傅里叶变换提取每个品类的主频率分量，捕捉主要的周期成分，以识别销售量数据中的周期性波动。其计算公式为：

$$\text{fft_coeff_1} = \sum_{j=1}^n s_{ij} \cdot e^{-2\pi i(k-1)(n-1)/n} \quad (5)$$

(4) 周期特征: autocorr_lag_1

autocorr_lag_1 反映每个品类销售量数据的短期依赖性，通过计算相邻天数 b 之间的自相关系数来识别短周期规律。其计算公式为：

$$\text{autocorr_lag_1} = \frac{\sum_{j=2}^n (s_{ij} - \text{mean}_i) (s_{i(j-1)} - \text{mean}_i)}{\sqrt{\sum_{j=1}^n (s_{ij} - \text{mean}_i)^2 \cdot \sum_{j=2}^n (s_{i(j-1)} - \text{mean}_i)^2}} \quad (6)$$

通过上述特征的构建，我们能够对各品类销售数据的整体水平、波动性、趋势、周期性及频域特征进行全面描述，为后续的聚类分析提供可靠的特征支持。

4.4.2 K-means 聚类算法的建立

K-means 算法是一种聚类算法，利用数据间的某种相似度将特征相似的样本自动归到一个类别中，实现对数据的聚类划分^[1]。其目的是将一组数据划分为 k 个不同的簇，使得同一簇内的数据相似度高，不同簇之间的相似度尽可能低。

其基本原理是：对于一群数据，确定 k 个中心点，此处可使用肘部法确定，先假设分别将 k 值设置为 1、2、3 等等一系列值，计算样本点到各个质心的距离，而后根据距离绘制折线图，找出图中变化最大的点的 k 值，即为此处选定的 k 值。而后根据 k 个中心点的特征值和样本点的特征值，计算各个样本点到中心点之间的距离，选择距离最近的中心点，将其归为一类。计算完所有样本点，分至不同簇中，而后得到了 k 个簇。对于这 k 个簇而言，显然之前给定的质心不再是现在簇的中心点了，因而需要重新计算每个簇的质心，即该簇中各个样本点的平均值，更新质心点后，再次计算各个样本点到质心之间的距离，继续分簇，并再次更新质心。不断重复这一过程，直到质心不再改变，或各个聚类中的样本点保持不变或误差平方和局部最小，即得到 K-means 算法最终的分簇结果。

本文利用 K-means 聚类算法对各品类销售量形成的时间序列组成的历史数据进行分类，从而达到同一类别在需求上的特征最为相似的目的。其基本步骤如下：

Step 1 从待聚类样本数据中随机选择 k 个样本数据 $\{\mu_1, \mu_2, \dots, \mu_k\}$ 作为初始聚类中心。

Step 2 计算每个样本到各个初始聚类中心的欧氏距离，选择距离最近的聚类中心形成 k 簇，并根据距离公式更新 k 簇，距离计算公式为：

$$d = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|_2^2 \quad (7)$$

式中， μ_i 为簇 C_i 的中心样本。

Step 3 对新的 k 簇重新计算聚类中心。聚类中心计算公式为：

$$\mu_i = \frac{1}{C_i} \sum_{x \in C_i} x \quad (8)$$

重复步骤 2、3 直到满足终止条件 $|\mu_{n+1} - \mu_n| \leq \varepsilon$ 或达到预设迭代次数，则算法终止。

K-means 算法将形态相似的分量进行聚类，随后将分类一致的分量进行相加合并，增强合并后分量的整体趋势，减少了噪声对整体趋势的影响，降低序列的复杂度，进而提高预测精度，同时因为合并导致分量数量减少，能够提高模型的训练和预测速度。[4]

4.4.3 各品类销售量序列聚类结果分析

利用 K-means 算法聚类后，序列数据不全为 0 的 343 个品类销售量序列在 22 年和 23 年分别被聚类为 3 类，部分类别聚类分析结果如图所示。

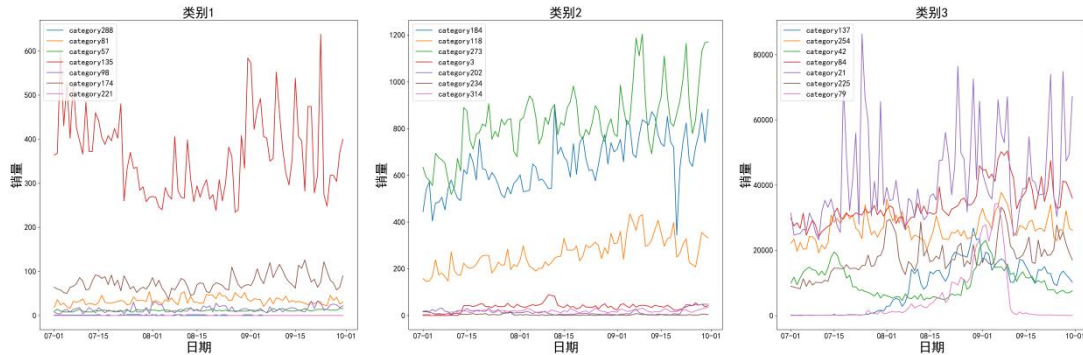


图 10 22 年部分品类销售数据聚类结果

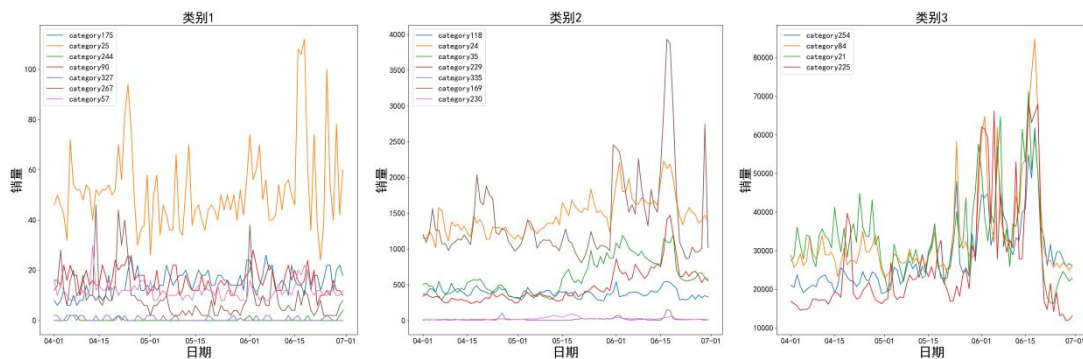


图 11 23 年部分品类销售数据聚类结果

从 2022 年的聚类结果来看，不同类别的销售量存在显著差异，部分品类表现出较高的销售量波动，而其他品类则较为平稳。同样，2023 年的聚类结果也

呈现出相似的分类特征，不同类别在销售量上具有明显的层次结构。通过比较 2022 年和 2023 年的聚类结果，可以发现部分品类在两个年份中的类别一致性较高，例如 2023 年类别三的品种在 2022 年类别三中都可以找到，表明这些品类的销售量波动具有一定的时间稳定性；而另一些品类的分类变化则提示其销售量可能受到外部因素影响，为后续预测提供了有价值的参考依据。

4.5 各品类库存量预测分析

为精准预测 2023 年 7 至 9 月各品类的库存量，本节采用指数平滑和 ARIMA 两种方法，以 2023 年 1 至 6 月的数据作为主要预测序列，结合 2022 年 7 至 9 月的数据进行辅助微调。首先，指数平滑通过对两段数据独立建模，提取各自的趋势和季节性特征，再根据时间接近性赋予 2023 年数据更高权重，最终融合两段模型的结果；其次，ARIMA 通过对 2022 年和 2023 年的数据分别建模，捕捉各自的趋势与周期特征，并以加权组合的方式融合预测结果。这种方式结合了最新趋势和历史同期模式，为 2023 年 7 至 9 月的库存预测提供了更可靠的依据。

4.5.1 指数平滑预测模型的建立

指数平滑是一种经典的时间序列预测方法，通过赋予较近数据更高权重，实现对趋势和季节性成分的平滑处理。在本问题中，采用三次指数平滑算法，以捕捉库存量数据的长期趋势、季节性波动和短期变化。三次指数平滑能够更好地适配本问题中跨年份的数据特征，通过对 2022 年 7 至 9 月和 2023 年 1 至 6 月数据分别建模，结合时间接近性赋予不同权重，从而生成对 2023 年 7 至 9 月库存量的精准预测。其基本步骤如下：

Step 1 对 2022 年 7 至 9 月和 2023 年 1 至 6 月的数据分别应用三次指数平滑，提取各自的趋势和季节性成分，构建两个独立模型：model_22 和 model_23。三次指数平滑的计算公式如下：

$$S_t^{(1)} = \alpha y_t + (1 - \alpha) S_{t-1}^{(1)} \quad (9)$$

$$S_t^{(2)} = \alpha y_t + (1 - \alpha) S_{t-1}^{(2)} \quad (10)$$

$$S_t^{(3)} = \alpha y_t + (1 - \alpha) S_{t-1}^{(3)} \quad (11)$$

Step 2 确定指数平滑权重。为实现平滑的预测过渡，对 2023 年 1 至 6 月数据赋予更高的平滑权重 α_2 ，对 2022 年 7 至 9 月的数据使用较低的平滑权重 α_1 。这种权重设置反映了 2023 年数据对预测月份的较强影响。具体权重参数设置为： $\alpha_2=0.7$ ， $\alpha_1=0.3$ 。

Step 3 计算趋势和季节性系数，分别获得 model_22 和 model_23 的趋势和季节性成分。趋势和季节性系数的计算公式如下：

$$a_t = 3S_t^{(1)} - 3S_t^{(2)} + S_t^{(3)} \quad (12)$$

$$b_t = \frac{\alpha}{2(1 - \alpha)^2} \left[(6 - 5\alpha) S_t^{(1)} - 2(5 - 4\alpha) S_t^{(2)} + (4 - 3\alpha) S_t^{(3)} \right] \quad (13)$$

$$c_t = \frac{\alpha^2}{2(1-\alpha)^2} [S_t^{(1)} - 2S_t^{(2)} + S_t^{(3)}] \quad (14)$$

Step 4 组合预测。以 model_23 的预测结果为主要参考，同时利用 model_22 的趋势成分进行微调。通过加权平均将两个模型结果融合，以得到综合预测值：

$$\hat{y}_t^{\text{final}} = \beta \cdot \hat{y}_t^{\text{model_23}} + (1 - \beta) \cdot \hat{y}_t^{\text{model_22}} \quad (15)$$

Step 5 生成最终预测值。通过组合的趋势和季节性成分，得到 2023 年 7 至 9 月的库存量预测结果：

$$Y_{t+T} = a_t + b_t T + c_t T^2 \quad (16)$$

其中， Y_{t+T} 表示未来第 T 时刻的预测值，通过三次指数平滑的组合预测，确保模型平滑地延续至目标月份，反映最新趋势和历史季节性的综合影响。

4.5.2 ARIMA 预测模型的建立

为实现对 2023 年 7 至 9 月各品类库存量的精准预测，本小节采用了 ARIMA 模型，通过分段建模分别对 2022 年 7 至 9 月和 2023 年 1 至 6 月的数据进行拟合。ARIMA 模型是一种广泛应用于时间序列预测的统计模型，能够捕捉数据中的趋势和季节性特征^[2]。在本模型构建过程中，以 2023 年 1 至 6 月的库存量数据作为主要预测序列，同时将 2022 年 7 至 9 月的同期数据作为辅助序列，以对预测结果进行微调。其基本步骤如下：

Step 1 对 2022 年 7 至 9 月和 2023 年 1 至 6 月的数据分别进行 ARIMA 建模，以捕捉每段数据的趋势和季节性特征。ARIMA 模型的基本形式为

$$y_t = c + \sum_{i=1}^p \phi_i y_{t-i} + \sum_{j=1}^q \theta_j \epsilon_{t-j} + \epsilon_t \quad (17)$$

其中， y_t 表示时间 t 的观测值， c 为常数项， ϕ_i 和 θ_j 分别表示自回归项和移动平均项的系数， ϵ_t 为误差项。通过分别对 2022 年和 2023 年两段数据建模，可以充分利用数据的季节性和趋势特征。

Step 2 确定 ARIMA 参数。针对每段数据，使用 AIC（Akaike 信息准则）和 BIC（贝叶斯信息准则）选择最优的 ARIMA 模型参数 (p, d, q) 。AIC 和 BIC 的计算公式如下

$$\text{AIC} = 2k - 2\ln(L) \quad (18)$$

$$\text{BIC} = \ln(n) \cdot k - 2\ln(L) \quad (19)$$

其中， k 为模型参数的数量， L 为似然函数的值， n 为数据点的数量。通过最小化 AIC 和 BIC 值，选择具有最佳拟合效果的模型结构。

Step 3 生成加权组合预测，同（X）式通过加权平均的方式，以 model_23 的预测结果为主要参考，同时利用 model_22 的趋势成分进行微调。通过加权平均将两个模型结果融合，以得到综合预测值的预测结果进行融合，由于 2023 年 1 至 6 月的数据较接近预测时间段，因此赋予其更高的权重，以确保最新数据在预测中占主导地位。

4.5.3 各品类销售量预测效果评价

基于两段时间序列结合的思想，本文采用指数平滑和 ARIMA 模型对 2023 年 7 至 9 月的各品类库存量进行了预测，部分品类预测效果可视化结果如下图所示。

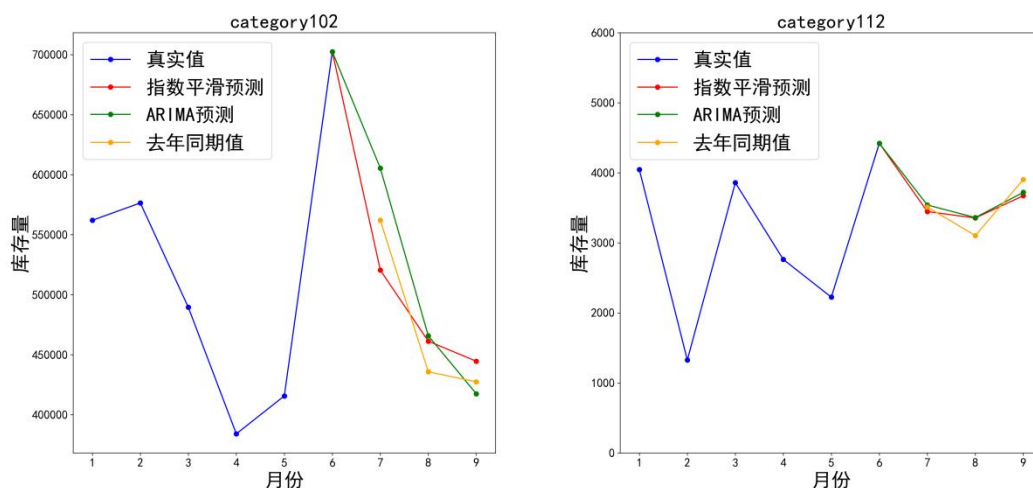


图 12 部分品类库存量预测效果图

从图中可以看出，指数平滑和 ARIMA 模型在不同品类的库存量预测中效果相近，均较好地学习了主要序列（2023 年 1 至 6 月）和辅助序列（2022 年 7 至 9 月）的特性。无论是 category102 还是 category112，两个模型的预测结果都能够准确跟随库存量的实际波动趋势，显示出对高峰和低谷的良好拟合效果，同时与去年同期的库存变化保持了一定的参考性。这表明两种模型不仅捕捉到了基准序列的核心趋势，还有效地融入了去年同期辅助序列的季节性特征，达到了预期的预测效果。因此，我们采用指数平滑和 ARIMA 模型的集成结果作为最终的预测结果，以期进一步提升预测的准确性和稳健性。

4.5.4 各品类库存量预测结果分析

基于本节的以 2023 年 4 至 6 月的库存量数据作为主要预测序列，2022 年 7 至 9 月的同期数据作为辅助序列的思想，采用指数平滑和 ARIMA 两种模型的集成结果对 2023 年 7 至 9 月的各品类库存量进行了预测，具体结果如下表所示。

表 2 月库存量预测结果

品类	七月库存量	八月库存量	九月库存量
category1	6655	6282	5946
category31	223	214	188
category61	238862	227160	217089
category91	14745	14618	14407
category121	81801	73929	76336
category151	130326	128350	131258
category181	259	285	331
category211	11875	10437	9438
category241	49501	43488	38241
category271	14660	14146	13425
category301	2473	2452	2857
category331	296	657	1090

从预测结果可以看出，各品类在 2023 年 7 至 9 月的库存量存在显著差异。部分品类如 category61 和 category151 的库存量保持在较高水平，且随着月份的推移呈现出逐渐减少的趋势，可能反映出这些品类的消耗速率较快。相比之下，

category31 和 category181 的库存量则维持在较低水平,表明需求或销售量较为平稳,库存消耗较慢。此外, category121 和 category241 的库存量在三个月内逐步下降,显示出季节性需求可能对其库存造成一定的压力;而 category301 和 category331 的库存量在九月有所上升,可能与预期的销售旺季或补货计划有关。整体来看,指数平滑和 ARIMA 两种模型提供的预测结果为库存管理和补货策略的制定提供了重要参考依据,有助于实现各品类库存的合理控制和优化。

4.6 各品类销售量预测分析

为继续准确预测 2023 年 7 至 9 月的各品类库存量,本节采用了 XGBoost、LightGBM 和 LSTM 三种模型,以实现和时间序列数据的多维度建模。具体方法是将 2023 年 4 至 6 月的数据作为主要预测序列,同时引入 2022 年 7 至 9 月的同期数据作为辅助序列,以微调模型的预测效果。XGBoost 和 LightGBM 作为基于决策树的集成学习模型,能够有效处理库存量数据中的非线性特征;而 LSTM 在捕捉时间序列中的长短期依赖方面具有优势。通过这三种模型的构建与优化,结合时间接近性和历史数据的趋势特征,可以实现对 2023 年 7 至 9 月库存量的精准预测。

4.6.1 XGBoost 预测模型的构建

在本问题中,XGBoost 被用于预测 2023 年 7 至 9 月的库存量数据。XGBoost 是一种基于梯度提升的集成学习模型,通过构建多个决策树并逐步优化残差来提升预测精度^[3]。为构建模型,我们以 2023 年 4 至 6 月的数据作为主要训练序列,并将 2022 年 7 至 9 月的数据作为辅助特征,帮助模型更好地捕捉季节性和趋势性特征。

模型的核心目标是通过最小化目标函数来优化预测效果。XGBoost 的目标函数可以表示为:

$$L(\phi) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (20)$$

其中, $L(\phi)$ 是线性空间的表达, $l(y_i, \hat{y}_i)$ 表示损失函数,用于衡量真实值 y_i 和预测值 $\hat{y}_i$ 之间的差异,常用平方误差作为损失函数。第二项为正则化项,用于控制模型的复杂度,避免过拟合。每棵树的结构复杂度 $\Omega(f_k)$ 可以表示为:

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (21)$$

其中, T 为树的叶子节点数, w_j 为叶节点的权重, γ 和 λ 是调节模型复杂度的超参数。

在模型训练过程中,我们首先对 2023 年 4 至 6 月的库存数据进行特征提取,包括日销售量、月累计量、库存变动率等,同时将 2022 年 7 至 9 月的同期数据作为辅助特征加入模型,以帮助模型识别季节性波动的影响。通过逐步优化每棵树,XGBoost 能够学习到库存量数据的复杂非线性关系,从而提升对未来月份的预测能力。

最终,模型通过逐步迭代,不断缩小预测值与真实值之间的误差,得到对 2023 年 7 至 9 月库存量的预测结果。XGBoost 的优越性在于其高效的计算速度和处理非线性特征的能力,使其在大规模数据和复杂时间序列预测中表现出色。

4.6.2 LightGBM 预测模型的构建

LightGBM 是一种高效的梯度提升框架，与 XGBoost 类似，通过构建多个决策树的序列来提升预测精度，但其采用的基于叶子节点的分裂策略（Leaf-wise）和高效的内存管理使其在速度和资源占用上更具优势^[4]。每次分裂时，LightGBM 选择带来最大增益的叶子节点进行分裂，从而提高建树效率。

在模型构建过程中，我们将 2023 年 4 至 6 月的数据作为主要训练集，同时将 2022 年 7 至 9 月的库存数据作为辅助特征，以捕捉库存量的季节性波动和趋势特征。模型的输入特征包括日销售量、月累计量、历史同期库存量、库存变动率等。通过这些辅助特征纳入模型，LightGBM 能够更有效地识别季节性和趋势性特征，从而提高对目标月份的预测精度。

在优化过程中，LightGBM 通过逐步迭代更新叶子节点的权重 w_j 以减小误差，更新公式如下：

$$w_j = w_j - \eta \frac{\partial l(y, \hat{y})}{\partial w_j} \quad (22)$$

其中， η 为学习率，用于控制每次更新的步长。通过这种逐步迭代的优化过程，LightGBM 能够逐渐减小预测误差，生成对 2023 年 7 至 9 月库存量的预测结果。LightGBM 的快速训练和低内存消耗使其在本问题的大规模数据预测中表现出显著优势。

4.6.3 LSTM 预测模型的构建

LSTM 是一种改进的循环神经网络（RNN），专门设计用于处理长时间序列数据，能够有效捕捉时间序列中的长短期依赖关系^[5]。与传统的 RNN 不同，LSTM 通过引入遗忘门、输入门和输出门等结构，缓解了长序列中常见的梯度消失和梯度爆炸问题，使其在复杂时间序列的预测中具有更好的表现。

在本模型中，使用 2023 年 4 至 6 月的数据作为主要训练序列，将 2022 年 7 至 9 月的同期数据作为辅助输入特征，以帮助模型捕捉销售量的季节性波动和趋势性变化。LSTM 模型将日销售量作为输入特征输入至 LSTM 的网络层进行时序特征学习。LSTM 单元计算具体原理如下：

遗忘门 $f(t)$ 通过当前的输入 x_t 和上一时刻的输出 h_{t-1} ，对上一时刻状态 C_{t-1} 中的元素设置权重，权重取值范围从 0 到 1。如式（X）所示：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (23)$$

新状态的更新由输入门 i_t 完成，输入门根据 x_t ， h_{t-1} 决定 C_{t-1} 状态的哪些部分写入当前时刻的状态 C_t 中去。如式（12）所示：

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (24)$$

同时，新状态的候选值 $\tilde{C}_t$ 由 $\tanh$ 函数产生，计算 $\tilde{C}_t$ 的公式如（13）所示：

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (25)$$

新状态 C_t 由 f_t 、 i_t 、 $\tilde{C}_t$ 、 C_{t-1} 共同决定，如式（14）所示：

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t \quad (26)$$

类似地，输出门 o_t 表示如式（15）所示：

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (27)$$

而 LSTM 最终的 h_t 是由输出门 o_t 与 C_t 决定的，如式（16）所示：

$$h_t = o_t \cdot \tanh(C_t) \quad (28)$$

模型在训练过程中，通过调整各个门的权重，使得 LSTM 能够捕捉库存量数据的长期趋势和周期性波动。在预测阶段，模型利用时间序列的历史特征进行多步预测，从而生成对 2023 年 7 至 9 月库存量的预测值。由于 LSTM 在长时间依赖关系中的表现优越，结合其对非线性特征的学习能力，LSTM 能够更准确地拟合库存量的变化趋势，是本问题库存量预测的有效模型选择。

4.6.4 各品类销售量预测效果评价

本文采用 XGBoost、LightGBM 和 LSTM 三种模型，基于 2022 年 7 至 9 月和 2023 年 4 至 6 月两段时间序列数据，对 2023 年 7 至 9 月的各品类销售量进行了预测。为评估各模型的预测效果，针对部分类别的训练过程进行了可视化，展示其训练集与测试集的拟合情况，具体效果如图所示。

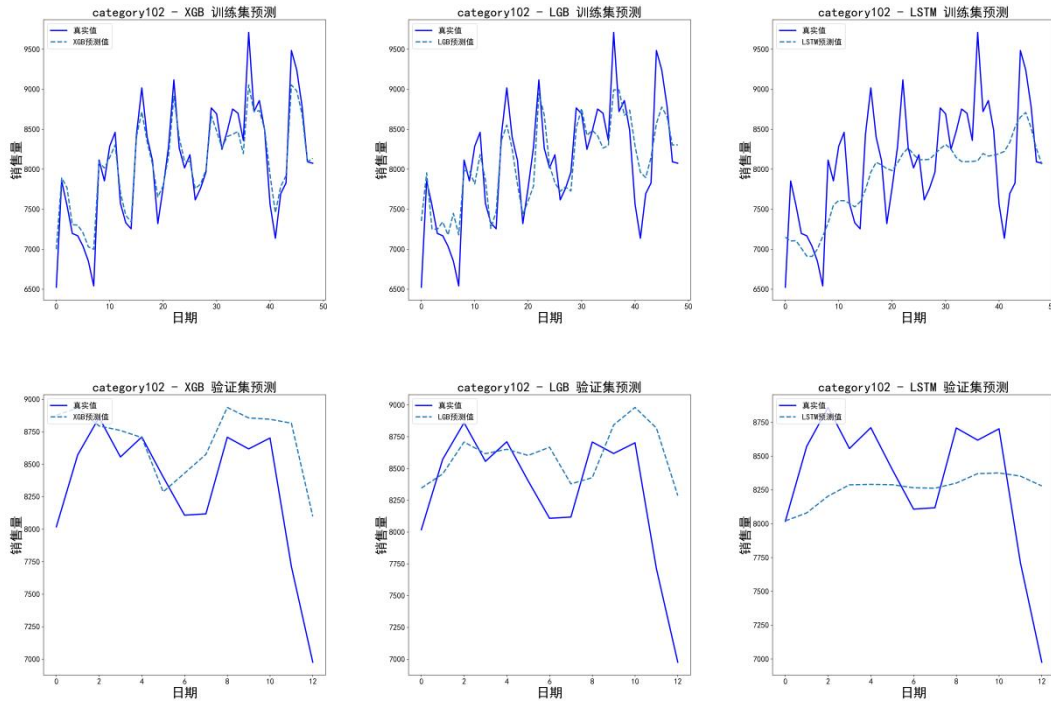


图 13 XGBoost、LightGBM 和 LSTM 对品类 102 的拟合与预测结果

从图中可以看出，XGBoost、LightGBM 和 LSTM 三种模型在训练集上的拟合效果总体较好，能够较为准确地捕捉销售量的波动趋势，表明模型在学习训练数据的特征方面表现出色。然而，在测试集上，不同模型的预测效果存在差异：XGBoost 和 LightGBM 在短期趋势预测上较为精准，但对于波动较大的数据点表现略显不足；LSTM 在长短期依赖关系的处理上具有优势，能够更好地拟合季节性波动，但在一些极值点上存在一定误差。这些结果表明，虽然各模型均能够对不同品类的销售量进行合理预测，但其适用性在不同波动特征下略有差异。

接下来，为了更加客观地评估 XGBoost、LightGBM 和 LSTM 三种模型在各品类销售量预测中的表现，本文采用了均方根误差（RMSE）、平均绝对百分比

误差（MAPE）和均方根标准化误差（RMSSE）等评估指标全面地分析各模型在不同品类销售量预测中的优劣，为后续模型优化提供数据支持。

（1）RMSE:

RMSE 是一种常用的误差评估指标，能够衡量模型预测值与实际值之间的平均误差大小。RMSE 对较大的误差较为敏感，因此能够反映模型在预测大幅度波动时的表现。较低的 RMSE 表示模型在预测中的误差较小，预测效果较好。RMSE 的公式为：

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (29)$$

（2）MAPE:

MAPE 用于衡量预测误差相对于实际值的百分比，能反映预测值偏离实际值的相对程度。MAPE 的单位为百分比，适合用于比较不同数据尺度下的预测效果。较低的 MAPE 值表示模型的相对误差较小，预测效果较为理想。MAPE 的公式为：

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\% \quad (30)$$

（3）RMSSE:

RMSSE 是一种标准化的误差评估指标，通过与历史数据的误差进行对比，能够评估模型在不同数据尺度下的表现。RMSSE 常用于对数据波动较大或不同尺度的数据进行一致性比较。RMSSE 值越小，表示模型在不同尺度下的误差越小，预测效果越稳定。RMSSE 的公式为：

$$RMSSE = \sqrt{\frac{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}{\frac{1}{n-1} \sum_{i=2}^n (y_i - y_{i-1})^2}} \quad (31)$$

本文采用了上述三个指标来衡量 XGBoost、LightGBM 和 LSTM 三种模型在品类销售量上的预测效果评价。部分品类及各品类平均的各模型评估结果如下表所示。

表 3 模型预测评估系数表

品类	Model	RMSE	MAPE	RMSSE
category101	XGB	428.0511632	19.49516048	0.043372592
	LGB	399.1755985	19.8097305	0.039394035
	LSTM	428.5563366	22.3791177	0.039785974
category111	XGB	164.1000872	19.978139682	0.026316206
	LGB	102.1919114	11.81408043	0.032458594
	LSTM	115.6683639	14.19637203	0.039416186
category121	XGB	380.498391	20.04316236	0.03416659
	LGB	355.2978288	19.19754157	0.03219206
	LSTM	414.9679277	21.54735105	0.038376401
category131	XGB	477.8878613	13.588139448	0.047265808
	LGB	385.4508948	13.881616684	0.03812327
	LSTM	629.7449345	16.605415977	0.06228533

各品类平均	XGB	258.8782856	18.154627244	0.023615567
	LGB	264.5389597	18.74300122	0.026862303
	LSTM	259.2957516	19.28829118	0.025091602

从表中可以看出，XGBoost、LightGBM 和 LSTM 三种模型在不同品类上的预测效果存在差异。总体来看，LightGBM 在大部分品类中表现出较低的 RMSE 和 MAPE 值，说明其在捕捉时间序列的趋势和季节性波动方面较为有效。XGBoost 在 RMSSE 指标上表现相对较好，表明其在整体上具有较高的预测稳定，而 LSTM 的预测效果在各项指标中均较为平均。各品类的平均结果显示，LightGBM 在三种模型中表现最佳，尤其在 RMSE 和 MAPE 方面误差较低，相比其他两个模型更适合作为本问题的销售量预测模型。

4.6.5 各品类销售量预测结果分析

基于本节的以 2023 年 4 至 6 月的销售量数据作为主要预测序列，2022 年 7 至 9 月的同期数据作为辅助序列的思想，采用 XGBoost、LightGBM 和 LSTM 三种模型对 2023 年 7 至 9 月的各品类销售量进行预测后的具体结果如下表所示。

表 4 日销量预测结果

品类	7.1	7.11	7.21	7.31	8.11	8.21	8.31	9.11	9.21
category1	24	26	25	26	27	26	26	25	25
category31	0	0	0	0	0	0	0	0	0
category61	1928	2260	2460	2291	2222	2501	2471	2255	2488
category91	1	3	4	1	2	2	2	2	2
category121	1012	1242	1352	1315	1354	1370	1363	1358	1344
category151	2155	2026	1973	1938	1951	1961	1976	1969	1981
category181	3	4	5	5	5	4	5	5	5
category211	19	24	22	26	19	24	27	23	24
category241	847	826	825	815	823	825	834	826	820
category271	166	160	150	149	148	145	155	146	150
category301	20	29	30	51	46	47	45	49	47
category331	484	496	533	524	521	526	518	507	527

从预测结果可以看出，不同品类在 2023 年 7 至 9 月的日销量表现出显著的差异。例如，category61 和 category121 的销量在预测期内保持高位，并呈现一定的波动性，可能受季节性需求或市场波动的影响。而像 category31、category91 和 category181 等品类的日销量则相对较低，甚至保持为零，表明这些品类在该时间段内的市场需求较小或处于销售淡季。此外，category151 和 category241 的销量波动相对平稳，变化幅度较小，表现出较高的稳定性，适合作为库存稳定供应的参考品类。这些预测结果给后续的库存优化策略提供了重要的量化依据。

五、问题二的建模与求解

本节针对“一品一仓”品类分仓优化问题，重点研究在限制仓库容量和产能的条件下，如何在最小化仓储成本的同时，实现各品类与仓库的最佳匹配。为了更科学、合理地分配仓库资源，本文构建了以品类关联度最大化、仓容利用率最大化、产能利用率最大化以及仓储成本最小化为目标的多目标优化模型。通过引

入改进的 NSGA-III 算法解决多目标优化问题，在原有 NSGA-III 的基础上，融入反向学习和动态交叉变异操作，以提高求解的全局搜索能力和多样性，从而实现更优的分仓决策。

5.1 问题分析

问题二的主要任务是根据品类和仓库的特性，确定各品类的最优仓库分配方案，以实现仓储成本最小化和仓库资源利用率最大化的目标。为此，本问题需建立包含多个目标函数的多目标优化模型，核心优化目标包括仓储成本的最小化、品类关联度的最大化，以及仓库容量和产能利用率的提升。为了确保每个品类在指定仓库中能够得到合理存放，同时满足“一品一仓”的特定约束，模型需平衡多个相互关联的目标，而仅通过单目标或加权方式难以准确评估不同目标的重要性和优先级。

在这种多目标优化场景下，NSGA-III（非支配排序遗传算法-III）作为一种典型的多目标优化算法，具备良好的收敛性和多样性。NSGA-III 引入了参考点分布，以帮助算法均匀地探索解空间，从而在各个优化目标之间取得平衡。然而，随着问题复杂性和优化维度的提升，NSGA-III 在解决多目标问题时可能出现收敛速度降低、易陷入局部最优等问题。为此，本文对 NSGA-III 进行改进，引入反向学习策略和动态交叉变异机制，以增强算法的全局搜索能力和种群多样性。反向学习策略可以在种群初始化和迭代过程中引导个体朝着未探索区域扩展，提升种群的多样性，从而降低算法陷入局部最优的可能性。动态交叉变异机制则通过自适应地调整交叉与变异概率，灵活应对种群在不同阶段的进化需求，进一步提升收敛效率和优化效果。

在问题二的建模与求解中，结合上述改进后的 NSGA-III 算法，使得模型能够在兼顾仓储成本、品类关联度、仓容和产能利用率的同时，实现更合理的资源分配和优化方案，从而满足“一品一仓”分配要求。

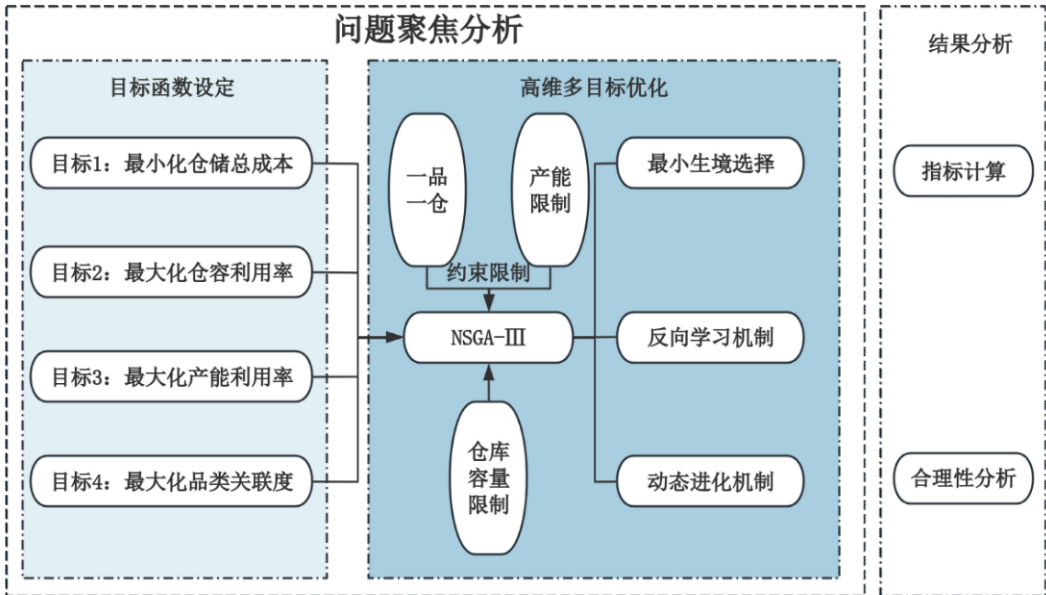


图 14 问题二解决方案整体框架

5.2 “一品一仓” 品类分仓规划模型的建立

本小节探讨了“一品一仓”品类分仓规划模型的构建方法，旨在通过合理分配各品类到不同仓库，实现仓储成本的最小化和仓库资源的高效利用。模型在满足“一品一仓”约束的前提下，考虑了仓库租赁成本、仓容利用率、产能利用率和品类关联度等关键因素，精确反映实际仓储管理需求，为企业制定符合实际的最优分仓方案提供科学依据。

5.2.1 决策变量的定义

在建立“一品一仓”品类分仓规划模型时，决策变量是模型的核心组成部分，用于描述每个品类在特定仓库中的存放情况。通过合理定义这些变量，本文可以量化每个品类在不同仓库中的存放方式，从而在实现仓储成本最小化和资源利用率最大化方面提供依据。

品类存放决策 x_{ij} :

x_{ij} 该变量为二元决策变量，用来表示品类 i 是否放置在仓库 j 中。若 $x_{ij}=1$ ，则表示品类 i 放置在仓库 j 中；若 $x_{ij}=0$ ，则表示品类 i 不放置在该仓库。该变量的引入可以有效控制各品类的仓库分配，使模型满足“一品一仓”的需求，从而实现仓库资源的优化配置。

5.2.2 约束条件的设定

本小节将详细介绍模型中的约束条件，这些条件确保了品类分仓决策的合理性和可行性。约束条件涵盖了“一品一仓”分配规则、仓库容量限制、仓库产能限制等关键因素，通过这些约束的设定，模型能够准确反映实际仓储管理的要求，为制定符合现实条件的最优分仓方案提供保障。具体而言，本小节构建了以下关键约束条件：

(1) 一品一仓约束

该约束条件确保每个品类只能分配到一个仓库，以保证“一品一仓”的分配要求。具体来说，对于每个品类，分配到的仓库数量之和必须等于1，即：

$$\sum_{j=1}^{140} x_{ij} = 1, \forall i \quad (32)$$

这一约束条件的引入是为了保证每个品类仅存放在一个仓库中，避免分散存储所带来的管理复杂性和成本增加，使模型更加符合实际的仓储管理策略。

(2) 仓库容量限制

该约束条件确保每个仓库的库存总量不超过其容量限制，以避免仓库超负荷运作。具体来说，对于每个仓库，所有分配到该仓库的品类库存量之和必须小于或等于仓库的最大容量，即：

$$\sum_{i=1}^{350} (x_{ij} \cdot s_i) \leq V_j, \forall j \quad (33)$$

其中， s_i 表示品类 i 的三个月库存总量， V_j 表示仓库 j 的容量。该约束条件的引入能够确保仓库资源合理分配，防止超载情况出现，从而保障仓库运作的稳定性和安全性。

(3) 仓库产能限制

该约束条件限制仓库的日出库量在其产能范围之内, 确保仓库的出货效率符合实际产能要求。具体来说, 对于每个仓库, 所有分配到该仓库的品类日销量之和不能超过仓库的产能上限, 即:

$$\sum_{i=1}^{350} (x_{ij} \cdot d_{it}) \leq P_j, \forall j, t \quad (34)$$

其中, d_{it} 表示品类 i 在日期 t 的日销量, P_j 表示仓库 j 的日出库产能上限。通过此约束条件, 模型可以有效避免仓库在出库方面超负荷运作, 提高仓库管理效率并保障服务质量。

通过以上约束条件的设定, 模型能够合理分配每个品类的存储仓库, 在满足“一品一仓”需求的同时, 有效利用仓库的容量和产能, 为优化分仓策略提供了坚实的约束基础。

5.2.3 目标函数的设定

在“一品一仓”品类分仓规划模型中, 目标函数的设定至关重要, 直接影响着模型的优化结果和实际仓储资源的利用效率。本模型的目标是在满足“一品一仓”要求的前提下, 通过合理分配各品类到不同仓库, 以最小化仓储总成本, 并最大化品类关联度、仓容利用率和产能利用率。

本节中目标函数的核心是综合最小化仓库租赁成本, 并最大化仓容和产能利用率。具体来说, 模型通过四个目标函数对仓库的租赁成本、品类关联度、仓容利用率和产能利用率进行优化。目标函数通过多重求和公式来涵盖每个品类和仓库在不同方面的表现, 以确保在最低成本下, 实现仓库资源的最优利用。

(1) 最小化仓储成本

该目标函数的目的是在“一品一仓”要求下, 通过合理选择仓库来减少总的仓储租赁成本。具体来说, 仓库的总租赁成本由分配到仓库的品类数量及其对应的仓库租赁费决定。仓库租赁成本部分公式为:

$$\min \sum_{i=1}^{350} \sum_{j=1}^{140} c_j \cdot x_{ij} \quad (35)$$

其中, c_j 表示仓库 j 的月租成本, x_{ij} 为二元决策变量, 表示品类 i 是否放置在仓库 j 。通过这个公式, 模型可以有效控制总仓储成本, 确保在最小成本下满足分仓需求, 使得仓储资源分配符合实际运营的经济性。

(2) 最大化品类关联度

该目标函数的目的通过将关联度较高的品类集中分配到同一个仓库中, 以实现品类关联度的最大化。具体而言, 对于每一对不同品类 (i, k) , 若品类 i 和品类 k 均分配到仓库 j , 则计入它们的关联度 a_{ik} 。这样能够有效地将具有关联性的品类集中在同一仓库, 减少仓储管理成本并提高仓库运营效率。目标函数公式如下:

$$\max \sum_{j=1}^{140} \sum_{i=1}^{340} \sum_{k=i+1}^{340} a_{ik} \cdot x_{ij} \cdot x_{kj} \quad (36)$$

其中, a_{ik} 表示品类 i 和品类 k 的关联度; x_{ij} 和 x_{kj} 为二元变量, 分别表示品类

i 和品类 k 是否分配到仓库 j 。该目标函数确保品类关联度较高的品类对尽量被分配到同一仓库，从而实现仓储资源的高效配置和管理优化。

(3) 最大化仓库利用率

为了提高仓库的使用效率，模型设置了最大化仓库利用率的目标函数。仓库利用率衡量了仓库空间的实际利用情况，即各品类库存总量在仓库容量内的占比。通过优化仓库利用率，模型力图实现仓库空间的高效使用，避免资源浪费。具体公式如下：

$$\max \sum_{j=1}^{140} \left(\frac{\sum_{i=1}^{350} (x_{ij} \cdot h_i)}{V_j} \right) \quad (37)$$

其中， h_i 为品类 i 的三个月总库存量， V_j 为仓库 j 的总容量。该目标函数确保仓库资源的合理利用，有助于实现库存量与仓库容量的最佳匹配，从而减少因仓库空间不足或闲置而导致的成本和资源浪费，提升仓储管理的整体效能。

(4) 最大化产能利用率

该目标函数确保仓库的日出库量能够满足需求并符合实际的出库产能。产能利用率衡量了仓库在满足日出库需求的前提下，其出库能力的使用情况。具体公式如下：

$$\max \sum_{j=1}^{140} \sum_{t=1}^{92} \left(\frac{\sum_{i=1}^{350} (x_{ij} \cdot d_{it})}{P_j} \right) \quad (38)$$

其中， d_{it} 表示品类 i 在日期 t 的日销量， P_j 表示仓库 j 的日出库产能上限。通过该目标函数，模型能够确保仓库的出库产能被充分利用，以避免因产能不足导致的配送延误或因产能过剩而造成的资源浪费。该目标函数的设立不仅提高了仓库的运行效率，还为电商运营提供了稳定的供应链支持。

最终，通过该目标函数的设定，模型能够根据各品类的库存量、仓库容量和租赁成本等因素。结合之前设定的各项约束条件，目标函数为企业提供了合理的分仓优化方案，在最小化仓储成本的同时，保障了仓库资源的高效利用。

因此，根据以上约束条件与目标函数建立的“一品一仓”分仓规划模型为：

$$obj. \begin{cases} \min \sum_{i=1}^{350} \sum_{j=1}^{140} c_j \cdot x_{ij} \\ \max \sum_{j=1}^{140} \sum_{i=1}^{340} \sum_{k=i+1}^{340} a_{ik} \cdot x_{ij} \cdot x_{kj} \\ \max \sum_{j=1}^{140} \left(\frac{\sum_{i=1}^{350} (x_{ij} \cdot s_i)}{V_j} \right) \\ \max \sum_{j=1}^{140} \sum_{t=1}^{92} \left(\frac{\sum_{i=1}^{350} (x_{ij} \cdot d_{it})}{P_j} \right) \end{cases} \quad s.t. \begin{cases} \sum_{j=1}^{140} x_{ij} = 1, \forall i \\ \sum_{i=1}^{350} (x_{ij} \cdot s_i) \leq V_j, \forall j \\ \sum_{i=1}^{350} (x_{ij} \cdot d_{it}) \leq P_j, \forall j, t \end{cases} \quad (39)$$

5.3 改进的NSGA-III算法的应用

NSGA-III 算法在多目标优化问题中应用广泛，但随着优化维度的增加，其

收敛性和多样性逐渐下降,这会影其在更复杂问题上的效果。为提升 NSGA-III 在高维多目标问题上的性能,本文引入了**反向学习策略**和**自适应进化机制**,旨在增强算法的搜索效率和解的多样性。反向学习策略通过拓宽搜索空间,避免了初始解陷入局部最优,而自适应进化机制则在进化过程中动态调整交叉与变异操作,提高了算法的收敛速度和全局优化能力。改进后的 NSGA-III 算法在本问题中的应用,有望在满足多个优化目标的同时,获得更优的解集分布与质量。基于本问题的改进的 NSGA-III 算法的具体步骤如下:

Step 1 种群初始化

(1) 编码方式:

在本模型中,使用整数编码方法,以便更有效地表示各个品类的仓库分配情况。每个个体的染色体长度为 340,表示 340 个品类的仓库编号分配。具体来说,染色体单位上每个品类 i 对应其分配到的仓库编号 j ,其中 j 的取值范围为 $j=1,2,\dots,140$ (对应仓库编号)。此限制确保每个品类只能分配到一个仓库,从而符合“一品一仓”的原则。

(2) 生成初始种群:

随机生成 N 个个体作为初始种群 P_k 。对于每个个体而言,为每个品类 i 随机分配一个仓库编号 j ,使其满足仓库容量和出库产能的约束条件。通过这种初始化方法,确保初始种群具有多样性和代表性,为后续的进化操作提供了良好的基础。

Step 2 反向学习机制

(1) 生成反向种群:

反向学习机制旨在通过生成反向种群,以增加搜索空间的覆盖度,增强种群的多样性^[7]。对种群 P_g 中的每个个体 $X_i = [x_{i1}, x_{i2}, \dots, x_{i340}]$,生成对应的反向个体 $X'_i = [x'_{i1}, x'_{i2}, \dots, x'_{i340}]$,其中每个变量 x_{ij} 的反向变量 x'_{ij} 计算公式为:

$$x'_{ij} = 140 + 1 - x_{ij} \quad (40)$$

该公式通过对仓库编号的选择进行反转,拓展了搜索的解空间,使得算法能够在较大的范围内进行全局搜索。引入反向学习机制的目的是填补对解空间的补充搜索,确保算法在初始阶段不会偏离全局最优区域。通过这种方式,反向学习机制能够增加种群的多样性,有助于避免算法在早期陷入局部最优,进而提升整体的搜索效率和收敛质量,为后续的优化步骤提供更为全面的搜索基础。

(2) 选择优良个体:

将初始种群 P_g 和反向种群 Q_g 合并,得到一个规模为 $2N$ 的种群 $R_g = P_g \cup Q_g$ 。然后,从 R_g 中选择适应度较高的 N 个个体,形成新的初始种群 P_g ,从而保证种群多样性和整体适应度水平。此操作能够平衡探索和开发,增强种群的全局优化能力。

Step 3 选择、交叉和变异操作

(1) 选择操作:

在种群中,根据适应度排序和非支配等级,选择表现较优的个体。优先选择非支配排序中等级较低的个体,以保留那些在多个目标上表现出色的解。该操作确保了精英解的存留,避免潜在良好解的丢失。

(2) 自适应交叉操作:

在进化过程中,为了保持种群多样性,交叉概率 P_{ci} 根据当前迭代次数 g 和个体适应度层级动态调整。具体公式如下:

$$P_{ci} = \begin{cases} 0.9, & g \leq \frac{G}{4} \\ 0.8, & \frac{G}{4} < g \leq \frac{G}{2} \\ 0.7, & \frac{G}{2} < g \leq \frac{3G}{4} \\ 0.6, & g > \frac{3G}{4} \end{cases} \quad (41)$$

其中， G 表示最大进化代数。随着代数 g 的增加，交叉概率逐渐降低，有助于在算法的早期阶段进行更多的全局搜索，以保证更广的搜索空间覆盖。进入后期阶段时，交叉概率的降低使得算法更聚焦于局部搜索，从而提升解的局部优化效果。

在交叉过程中，采用部分匹配交叉（PMX）方法，将两个父代个体的部分基因序列进行交替交换，生成子代个体 X_{off} 。此种交叉方式既能确保种群解的多样性，又能在仓库分配方案上满足仓容和产能限制，从而优化算法的求解合理性。随着代数增加，交叉概率逐步降低，有助于在早期阶段进行更多探索，后期更关注解的局部优化。

（3）自适应变异操作：

变异概率自适应调整：变异概率 P_{mi} 也根据代数 g 动态调整，以确保在早期阶段更大程度地探索搜索空间。具体公式如下：

$$P_{mi} = \begin{cases} 0.01, & g \leq \frac{G}{4} \\ 0.02, & \frac{G}{4} < g \leq \frac{G}{2} \\ 0.03, & g > \frac{G}{2} \end{cases} \quad (42)$$

在进化的早期阶段，较低的变异概率可以防止种群解质量的大幅波动，同时尽量保持解的多样性。随着进化过程的推进，变异概率逐渐增加，有助于种群跳出局部最优陷阱。在变异操作中，对个体中品类的仓库分配进行小范围变动，从而进一步探索搜索空间，确保解的合理性和符合仓容与产能的要求。

Step 4 基于均匀参考点的最小生境选择机制

（1）归一化目标函数值：

在多目标优化问题中，由于各个目标的量纲和数值范围不同，直接对各目标进行求解可能导致偏差。因此，为了消除不同目标之间的尺度差异，首先对每个目标函数进行归一化处理。具体而言，首先计算当前种群每个目标函数的最小值，并将其定义为理想点 z_j^{min} ，其中 $j = 1, 2, \dots, M$ 代表多目标中的每一个维度。然后，通过减去每个目标的最小值，得到去除偏差后的归一化目标值：

$$f'_j(x) = f_j(x) - z_j^{min} \quad (43)$$

这样一来，所有个体的目标值在归一化后都相对于该维度的最小值进行评估，使得各目标在同一尺度上进行优化。该归一化步骤能够有效减少目标之间的偏差影响，确保每个目标在后续选择过程中的公平性。

（2）计算目标偏量：

为了进一步评估每个个体与理想点的距离，本文采用了“成分最大值加权”策略，定义目标偏量 $ASF(x, w)$ 来度量个体与理想点的距离。通过引入权重向量 w ，我们可以在目标空间中对每个目标的偏差进行加权，以突出不同目标的重要性，

具体计算公式为：

$$ASF(x, w) = \max_{j=1, \dots, M} \left(\frac{f'_j(x)}{w_j} \right) \quad (44)$$

其中， w_j 为目标 j 的权重，反映该目标在优化中的优先级。通过最大化加权后的目标值，能够确保每个个体在目标空间中保持合理的分布，避免在优化过程中因个别目标占据主导而导致其他目标被忽略。这一步骤的结果是，确定出距离目标理想点最近的个体，并为后续选择提供依据。

(3) 选择超平面的截距点：

在归一化和偏量计算的基础上，构建目标空间的超平面，以确定每个目标维度的截距点 y_j ，其中 $j = 1, \dots, M$ 。此步骤的目的是在目标空间中为每个目标设置边界，以便控制其在整个种群中的分布。这一过程确保在多目标优化时，所有目标的边界均得到合理定义，从而避免了目标权重分配不均的问题，进一步保证解的多样性和公平性。

(4) 最小生境选择机制：

具体而言，最小生境选择机制旨在通过均匀分布个体到各个均匀参考点上，从而保证解在目标空间中的多样性。该过程的核心在于确保所有解都能在多目标空间中均衡分布，避免解集中于某一特定区域，提高了解的多样性。

Step 5 生成下一代种群并判断终止条件

(1) 生成下一代种群：

从非支配排序和最小生境选择中选出的 N 个个体作为下一代种群 P_{g+1} ，保证解集逐步收敛至最优区域。

(2) 判断终止条件：

设置了两种终止条件：若达到最大迭代次数 G ，或种群适应度变化低于某阈值，则停止进化，并输出最终非支配解集，获得在四个目标上均衡分布的帕累托前沿解。最终解集为最小化仓储成本、最大化品类关联度、最大化仓容利用率和最大化产能利用率的优化组合。改进的 NSGA-III 算法的流程如图 8 所示：

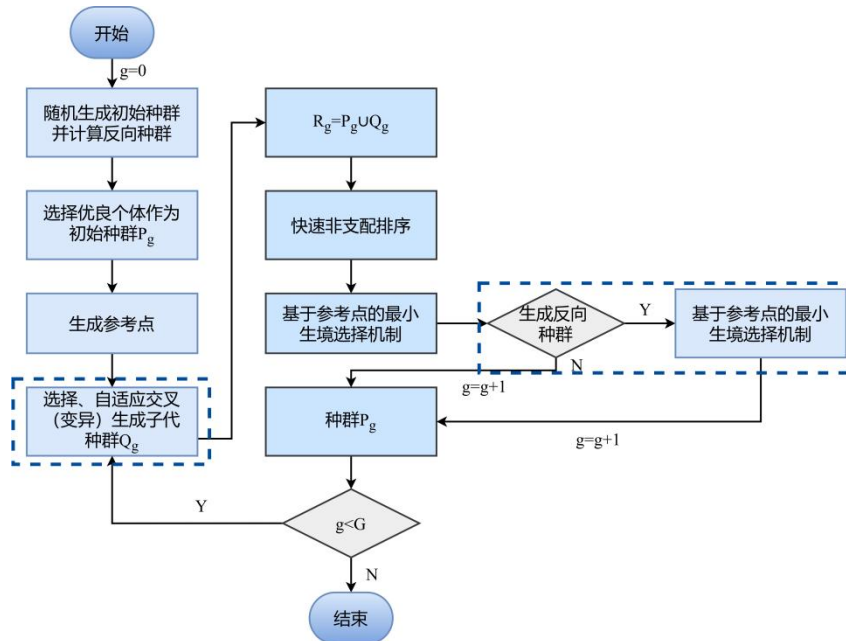


图 15 改进的 NSGA-III 算法流程图

5.4 “一品一仓” 品类分仓规划模型的求解

5.4.1 算法迭代结果分析

本节我们展示了经过 500 次迭代的标准的 NSGA-III 算法与改进后的 NSGA-III 算法在成本、仓容利用率、产能利用率以及品类关联度等多个目标上的收敛情况。为了更直观地比较这两种算法的性能，我们从各个目标的变化趋势入手，评估其在优化效果上的差异。

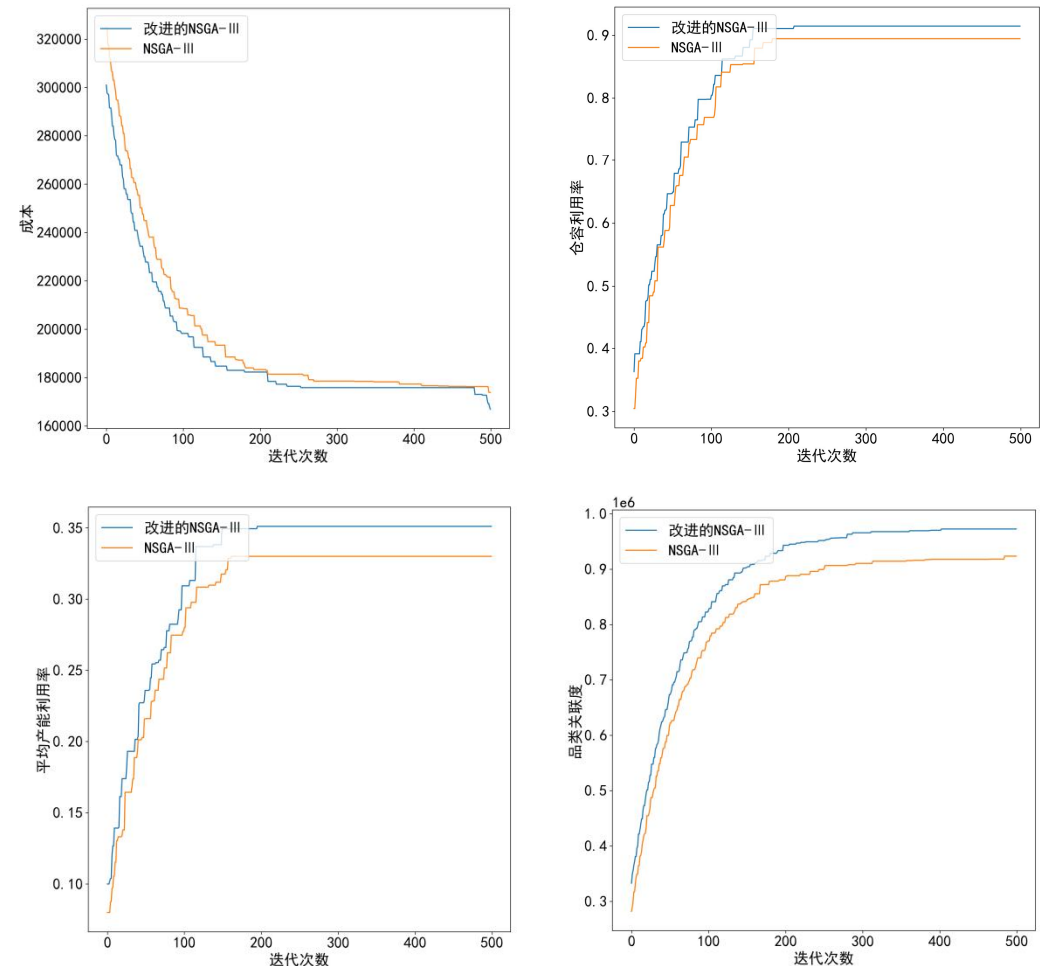


图 16 算法迭代结果对比图

从图中可以看出，改进后的 NSGA-III 算法在各项指标上均表现出更优的收敛性能。首先，在成本方面，改进后的算法在更少的迭代次数内达到了更低的成本值，相比标准的 NSGA-III 算法，表现出更快的收敛速度和更优的最优解。同时，在仓容利用率和产能利用率上，改进算法的提升也较为明显，仓容利用率迅速逼近 0.9，平均产能利用率也稳定在 0.35 左右，均优于标准 NSGA-III。此外，在品类关联度方面，改进算法同样表现更佳，达到了更高的最优值。这表明改进后的算法在整体资源分配和利用率提升方面有明显的优势。

综合来看，改进后的 NSGA-III 算法在多目标优化上实现了更高的效率和效果，其更优的收敛性使得各项资源指标得到了更充分的优化。该算法不仅加快了最优解的收敛速度，而且提升了解的质量，具有更好的应用价值。

5.4.2 “一品一仓” 品类分仓结果分析

表 5 “一品一仓” 品类分仓结果

品类	仓库
category1	32
category31	94
category61	55
category91	66
category121	99
category151	27
category181	99
category211	39
category241	45
category271	108
category301	99
category331	130

在“一品一仓”分仓策略下，每个品类被分配到一个特定仓库，以实现品类的集中存储。由分仓结果可以看出，不同品类的分布充分考虑了仓库的容量、品类的关联度和各仓储资源的使用效率。例如，category1 被分配到仓库 32，category61 被分配到仓库 55，而 category301 和 category181 则共同分配在仓库 99。这种分配方式在满足“一品一仓”需求的同时，有助于减少仓库内的管理复杂性，实现资源的高效利用，并为后续库存管理和供应链优化提供了更加稳定的支持。

5.4.3 “一品一仓” 品类分仓业务指标分析

在“一品一仓”策略下，我们对各仓库的业务指标进行了统计和分析，以评估不同仓库在品类关联度、平均产能利用率以及仓容利用率方面的表现。表 X 展示了部分仓库的业务指标统计数据，为仓储资源分配的有效性和各项业务指标的优化提供了数据支持。这些指标可以帮助我们更清晰地了解各仓库的资源使用效率和管理效果，从而为后续的优化策略制定提供参考依据。

表 6 部分仓库 “一品一仓” 品类分仓业务指标统计

仓库编号	品类关联度	平均产能利用率	仓容利用率
13	61324	0.499252598	0.988930565
18	2296	0.485289701	0.994570732
20	3660	0.267319501	0.934695727
24	3203	0.618521584	0.999976172
27	9472	0.59594497	0.982354405
32	1776	0.371396761	0.821120529
33	7045	0.472652388	0.995980695
40	16097	0.482626938	0.893192332
45	2092	0.2269457	0.836590804
49	1321	0.328961739	0.867165982
84	4320	0.259327017	0.971278506
99	14522	0.349074001	0.878042604
108	6968	0.116793518	0.822522061

116	6786	0.562984256	0.957769289
139	133234	0.900213564	0.880629248

从表中的数据来看，不同仓库在品类关联度和资源利用率上表现出显著的差异。仓库编号 139 的品类关联度最高，达到了 133234，且产能利用率和仓容利用率分别为 0.900 和 0.881，显示了较高的业务效率。相对而言，仓库编号 108 的产能利用率最低，仅为 0.117，但其仓容利用率也达到了 0.823，说明其空间利用率尚可，适合低频出库的存储需求。整体而言，这些指标反映出“一品一仓”策略在满足不同品类需求、优化仓储资源分配方面的有效性。

六、问题三的建模与求解

在本节我们的主要工作是构建适应“一品多仓”场景的分仓规划模型，并优化其在仓库容量与产能等多项指标下的资源配置。首先，我们针对每个品类允许存放于不多于三个仓库的限制，设置了相应的决策变量，并引入了各类关键的约束条件，以确保仓库资源得到最合理的利用。随后，考虑到品类关联度在实际运营中的重要性，本节将其作为主要优化目标，以实现同件型和高级品类的集中存放，从而在管理成本和分仓复杂度上实现双重优化。

6.1 问题分析

问题三的核心在于从“一品一仓”分仓模式扩展至“一品多仓”场景中，针对多个仓库的分配和关联品类的集中存放问题进行优化。。因此，问题的关键在于在多个优化目标下，通过精确的分仓决策达到仓库资源的最佳配置。为解决这一问题，模型设置了对关联度的优先优化，同时引入了品类的仓库分布比例一致性、仓库容量限制、日出库产能等多项约束，以确保各项指标能够在复杂的仓储场景下得以满足。分仓方案中还增加了品类关联度、产能利用率、仓容利用率等业务指标，并通过决策偏好机制对这些指标赋予动态加权，从而实现多目标优化中的差异化优先级。

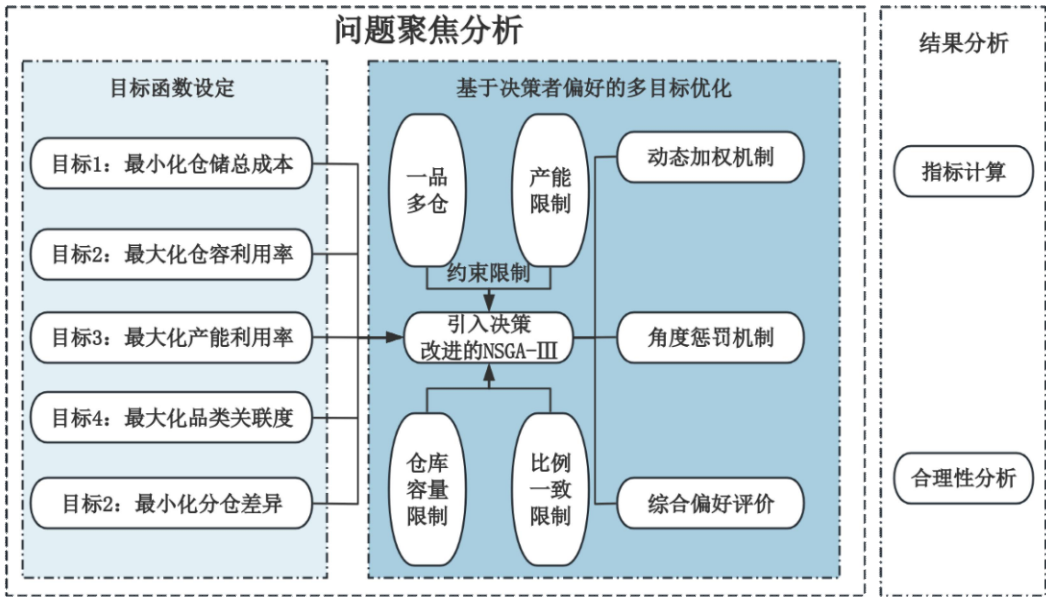


图 17 问题三解决方案整体框架

6.2 “一品多仓” 品类分仓规划模型的建立

本小节构建了“一品多仓”品类分仓规划模型，在延续第二问“一品一仓”模型的基础上，进一步优化了仓储资源的配置策略。针对多仓协作的场景，该模型不仅考虑了每个品类在多个仓库中的分配情况，还特别引入了库存和销量在多仓库之间的分配比例控制，以便于更灵活地管理各品类的库存和日出库需求。同时，模型通过增加关联性优化目标，以同属性品类的集中分配为导向，在资源使用效率最大化的前提下，减少分仓分散度带来的管理复杂性和成本。通过新增的决策变量、改进的约束条件以及优化的目标函数，模型力求在满足实际仓储需求的同时，实现仓储资源的高效配置与成本优化。

6.2.1 新增决策变量的定义

在问题二的基础上，根据问题三的要求本节的“一品多仓”分仓规划模型在决策变量的设定上考虑了每个品类在多仓库（不多于三个）的分配情况，还引入了多仓之间的库存量和销售量分配需求。这一设定使得模型能够在不同仓库之间进行更加灵活的库存和出库管理，从而提升仓库的整体利用效率。为此，本节新增了以下决策变量：

（1）品类库存分仓决策 y_{ij} ：

该变量为连续变量，用于表示品类 i 在仓库 j 中的库存量。通过控制该变量，模型可以精准地分配各品类在不同仓库的库存量，确保符合仓库容量的限制，同时实现资源的高效利用。

（2）品类日销售分仓决策 d_{ijt} ：

该变量为连续变量，表示品类 i 在仓库 j 中第 t 天的销售量。该变量的定义使得模型能够控制各品类的日出库需求，确保符合各仓库的出库产能限制，从而保障仓储运营的稳定性。

（3）品类分配比例 r_{ij} ：

该变量为连续变量，用来表示品类 i 在仓库 j 中的分配比例。通过控制该变量，模型可以更细化地分配每个品类在不同仓库中的比例，从而确保在满足各仓库存储与出库能力的前提下，实现资源的最佳配置。这一变量的引入，使得模型能够在仓库间分配库存量时更加精确，避免资源浪费，并提升整体仓储和物流效率。

通过以上决策变量的引入，模型不仅可以灵活地描述多仓储环境中的品类存放情况，还能在确保仓容和产能限制的前提下优化品类的分仓方案。这些决策变量的合理定义为后续模型的目标函数和约束条件提供了精确的量化基础。

6.2.2 约束条件的增设及改进

本小节详细介绍了第三问模型中的约束条件，以确保仓储分配决策的合理性和可行性。不同于第二问，此部分的约束条件进一步考虑了仓库的容量和产能限制，以及品类库存和销售量在各仓库的分布比例，使得模型更加贴合实际的仓储管理需求。这些约束条件的设定，帮助模型精确反映了仓储运作中的关键因素，使得分仓方案在满足运营要求的同时，实现资源的优化配置。具体而言，本小节构建了以下关键约束条件：

（1）单品多仓数量限制

为控制仓库的分配数量和降低管理复杂性，模型对每个品类的分仓数量进行限制，即每个品类最多只能分配至三个仓库。此条件不仅能减少仓库间的协调成本，还能防止分仓数量过多导致的管理难度提升。具体公式如下：

$$\sum_{j=1}^{140} x_{ij} \leq 3, \forall i \quad (45)$$

该约束条件确保了仓库资源的合理配置，使得分仓策略既符合运营需求，又保持了分仓方案的简洁性和高效性。这种限制能够帮助企业优化仓储配置，减少操作成本，进一步提升仓储管理的整体效率。

综上所述，通过仓库容量限制、产能限制、库存与销量分配比例一致性、以及单品多仓数量限制等关键约束条件的设定，模型可以有效保障仓储资源的合理利用。

(2) 仓库容量限制

仓库容量限制约束确保每个仓库的总库存量不会超过其最大容量，同时保证各品类的库存量在所有仓库之间的合理分配。具体来说，设定每个仓库的容量上限 V_j ，品类 i 在仓库 j 中的库存量为 y_{ij} 。根据实际仓储要求，公式如下：

$$\sum_{i=1}^{340} y_{ij} \leq V_j, \forall j \quad (46)$$

此条件约束确保每个仓库的总库存不会超过其物理限制，从而避免仓库溢满或超负荷运行的问题，进而保障仓库内的物品安全和管理稳定性。

此外，还需确保每个品类的三个月总库存量在所有仓库间的分布之和等于其指定的总库存量。通过这一设定，可以有效管理品类的库存，使其符合实际库存需求并保持整体供应稳定。具体表达式为：

$$\sum_{j=1}^{140} y_{ij} = s_i, \forall i \quad (47)$$

其中， s_i 表示品类 i 的三个月总库存量。通过这一约束条件，模型能够确保库存分布的合理性，防止库存分配不均或分配超出仓储容量的情况。

(3) 仓库产能限制

仓库的日出库产能是衡量仓库运作效率的关键指标。设 P_j 为仓库 j 的每日出库产能上限，品类 i 在仓库 j 中日期 t 的销售量为 d_{ijt} ，该约束条件确保每个仓库的日出库量不会超过其产能上限。通过严格的产能限制，模型可以保证仓库出库需求的满足，避免因产能不足而导致的延误或库存积压。具体公式如下：

$$\sum_{i=1}^{340} d_{ijt} \leq P_j, \forall j, t \quad (48)$$

该条件有效约束了仓库的日出库能力，使模型在满足市场需求的同时，能够避免仓储产能不足所带来的延误风险。这一约束确保了仓库在出库效率上的优化，进一步支持电商企业的供应链稳定性。

此外，为保持品类销售量的合理分布，还规定每个品类的每日销量在所有仓库的分布比例之和必须等于其每日总销售量。公式如下：

$$\sum_{j=1}^{140} d_{ijt} = d_{it}, \forall i, t \quad (49)$$

其中， D_{it} 表示品类 i 在日期 t 的总预测销量。这一设定确保了模型中每日销售量的合理性和一致性，使库存分配能精确匹配实际市场需求。

(4) 库存和销量的分配比例一致

在仓储分配管理中，库存和销量的比例一致性是至关重要的。为确保同一品类在不同仓库中的库存量和销量的分布比例一致，模型引入分配比例变量 r_{ij} 。这一变量反映了品类 i 在仓库 j 中的分配比例，使得库存量和销量能够保持统一的分布模式。具体约束表达式如下：

$$y_{ij} = r_{ij} \cdot s_i, \forall i, j \quad (50)$$

$$d_{ijt} = r_{ij} \cdot D_{it}, \forall i, j, t \quad (51)$$

其中， s_i 为品类 i 的三个月总库存量， D_{it} 为品类 i 在日期 t 的总预测销量。这一约束条件确保了同一品类在不同仓库的库存和销量之间保持相同的分配比例，有助于仓库资源的平衡管理，避免出现因仓库间分配不均而导致的资源浪费或供需错配现象。

值得注意的是，分配比例 r_{ij} 仅在 $x_{ij} = 1$ 时非零，即只有当品类 i 确定分配至仓库 j 时，其分配比例才生效。因此，还需引入以下约束条件以控制分配比例变量的范围：

$$r_{ij} \leq x_{ij}, \forall i, j \quad (52)$$

通过这一限制，模型进一步确保了分配比例的一致性，从而增强了仓储管理的灵活性和精确性。

6.2.3 目标函数的增设与改进

在延续第二问“一品一仓”分仓模型的基础上，第三问的“一品多仓”模型进一步优化了品类间的关联性配置。在考量品类的高级品类和件型的基础上，第三问的模型力求将具有相同高级品类或件型的品类集中在同一仓库中，以减少分散带来的管理复杂度。同时，第三问在第二问的基础上对最大化仓容利用率和产能利用率的目标函数上进行了改进，从而提升整体资源的使用效率。

(1) 同高级品类或件型分仓

此目标函数旨在通过最小化同属性品类分散到不同仓库的情况，提升仓储资源的整合效率。与第二问不同，本节特别引入了同属性品类的配对变量 s_{ik} ，以衡量相同高阶品类或属性下的品类是否集中存放在同一仓库，从而在分仓规划中实现更合理的布局。

具体而言，目标函数通过求和公式来量化相同属性的品类分配在不同仓库的次数，以减少分散带来的管理复杂性。其表达式如下：

$$\min \sum_{j=1}^{140} \sum_{i=1}^{340} \sum_{k=i+1}^{340} h_{ik} \cdot |x_{ij} - x_{kj}| \quad (53)$$

其中 h_{ik} 为二元变量，表示品类 i 和品类 k 是否具有相同的高阶品类或属性。当两者相同时， $h_{ik}=1$ ；否则， $h_{ik}=0$ 。 $|x_{ij} - x_{kj}|$ 表示品类 i 和品类 k 在仓库 j 中的分配情况，若分配在不同仓库则此项为 1，否则为 0。

通过最小化该项，模型鼓励相同属性的品类集中于同一仓库，从而提升管理效率和资源利用率。本设计有助于减少分仓操作中的品类分散度，达到集约化管理的目的。相比于“一品一仓”模型，本节在保留基础分仓逻辑的前提下进一步优化了关联性配置，适用于多仓协作的场景。

(2) 最大化仓容利用率

仓容利用率的目标是通过优化仓库库存接近容量上限来提高仓库的使用效

率。具体来说，仓容利用率最大化的目标函数可以定义为：

$$\max \sum_{j=1}^{140} \left(\frac{\sum_{i=1}^{350} y_{ij}}{V_j} \right) \quad (54)$$

其中， y_{ij} 表示品类*i*在仓库*j*中的库存量， V_j 为仓库*j*的容量上限。该目标函数旨在最大化仓库的总库存量与其容量上限的比值之和，使仓库空间得到充分利用，减少闲置空间的浪费，从而在满足库存需求的同时有效降低仓储成本。

(3) 最大化产能利用率

产能利用率的目标是确保仓库的出库产能能够满足需求并接近仓库的实际生产能力上限。该目标函数的数学表达如下：

$$\max \sum_{j=1}^{140} \sum_{t=1}^{92} \left(\frac{\sum_{i=1}^{350} (x_{ij} \cdot d_{ijt})}{P_j} \right) \quad (55)$$

其中， d_{ijt} 表示品类*i*在仓库*j*中日期*t*的出库量， P_j 为仓库*j*的出库产能上限。通过最大化仓库每日出库量与产能上限的比值，模型可以确保仓库的出库能力被充分利用，有效避免产能不足导致的资源浪费，提升仓库的整体运营效率。

通过上述目标函数的设定，第三问的“一品多仓”分仓模型在延续第二问的基础上，进一步增强了关联性、仓容和产能利用等方面的优化。在兼顾每个品类的仓储需求和管理便捷性的同时，模型不仅确保了仓库的高效利用率，还通过集中同属性品类的分配策略降低了管理复杂度。这一模型的优化方案为实现物流成本的节约和运营效率的提升提供了科学支持。

因此，根据以上约束条件与目标函数建立的“一品多仓”分仓规划模型为：

$$\begin{aligned} \text{obj.} \left\{ \begin{array}{l} \min \sum_{i=1}^{350} \sum_{j=1}^{140} c_j \cdot x_{ij} \\ \max \sum_{j=1}^{140} \sum_{i=1}^{340} \sum_{k=i+1}^{340} a_{ik} \cdot x_{ij} \cdot x_{kj} \\ \max \sum_{j=1}^{140} \left(\frac{\sum_{i=1}^{350} (x_{ij} \cdot s_i)}{V_j} \right) \\ \max \sum_{j=1}^{140} \left(\frac{\sum_{i=1}^{350} (x_{ij} \cdot d_{it})}{P_j} \right) \\ \min \sum_{j=1}^{140} \sum_{i=1}^{340} \sum_{k=i+1}^{340} h_{ik} \cdot |x_{i,j} - x_{k,j}| \end{array} \right. \quad \text{s.t.} \left\{ \begin{array}{l} \sum_{j=1}^{140} x_{ij} \leq 3, \forall i \\ \sum_{i=1}^{340} y_{ij} \leq V_j, \forall j \\ \sum_{j=1}^{140} y_{ij} = s_i, \forall i \\ \sum_{i=1}^{340} d_{ijt} \leq P_j, \forall j, t \\ \sum_{j=1}^{140} d_{ijt} = d_{it}, \forall i, t \\ y_{ij} = r_{ij} \cdot s_i, \forall i, j \\ d_{ijt} = r_{ij} \cdot D_{it}, \forall i, j, t \\ r_{ij} \leq x_{ij}, \forall i, j \end{array} \right. \quad (56) \end{aligned}$$

6.3 引入决策者偏好评价的改进 NSGA-III 算法

问题三强调了将最大化品类关联度的目标作为优先优化方向，因此这是一个典型的基于决策者偏好的多目标优化问题。单纯地通过提高目标权重来实现偏好

引导可能存在若干问题，固定高权重可能导致算法过度偏向某一目标，而忽略其他目标的优化需求，使解的多样性和全局性受限。为此，我们需要引入动态偏好导向机制，以在优化过程中更好地平衡各个目标，增强算法在高维多目标优化问题中的收敛性和解的多样性。

针对上述问题，本文在第二问的 NSGA-III 基础上引入了动态加权因子、角度惩罚机制和综合评价函数。通过动态加权因子，算法可以根据迭代进程灵活调整对不同目标的偏好程度，以兼顾初期的偏好导向和后期的多目标均衡。此外，角度惩罚机制的引入帮助算法在多目标空间内维持解的偏好导向，从而避免解过度偏离优先目标方向。综合评价函数则进一步将动态权重和惩罚项整合为单一评价标准，确保偏好导向的解得到优先选择。这些改进不仅提升了 NSGA-III 算法在复杂多目标问题中的适用性，还增强了优化结果在优先目标上的收敛效果。具体的改进步骤如下：

Step 1 动态偏好加权因子的构建

(1) 动态加权因子

为了在不同优化阶段实现灵活的目标偏好控制，我们引入了动态加权因子 $\lambda_m(t)$ ，使得各目标的权重能够随迭代进程而变化。动态加权因子的核心是通过随时间调整偏好方向，从而在初期优先探索特定目标（如品类关联度的最大化），后期逐步向多目标均衡解转移。这样的设定避免了固定权重可能导致的搜索方向单一问题，使得算法既能在初期集中搜索优先目标，又能在后期向全局最优解靠拢。动态加权因子的设定让算法更具弹性，适应不同阶段的搜索需求，提高解的全局性和多样性。

(2) 加权因子函数

加权因子函数用于控制偏好目标权重的动态变化。通过以下公式逐步降低偏好目标的权重，使其由集中向分散调整：

$$\lambda_2(t) = \lambda_{\max} \cdot \left(1 - \frac{t}{T}\right) + \lambda_{\min} \cdot \frac{t}{T} \quad (57)$$

其中 $\lambda_{\max}$ 为初始偏好权重， $\lambda_{\min}$ 为最低权重， T 为最大迭代次数。动态加权因子的引入有助于在初期引导解向偏好目标方向发展，后期则平衡各目标的优化需求，确保算法收敛效果。

(3) 偏好引导加权目标函数

通过偏好引导加权目标函数，每个解的目标值 $f_m^\lambda(X_i)$ 可以按不同目标的加权因子重新计算，具体为：

$$f_m^\lambda(X_i) = \lambda_m(t) \cdot f_m(X_i) \quad (58)$$

此公式确保了偏好目标的导向作用，同时在动态调整过程中为其他目标提供适当权重，提升了算法的整体平衡性。

Step 2 角度惩罚机制的引入

(1) 偏好方向向量的定义

在多目标优化中，为了更精确地控制解的偏好方向，我们引入偏好方向向量 $\vec{u}_{\text{pref}}$ ，使算法能够明确优化的优先方向^[8]。若最大化品类关联度（目标 f_2 ）为偏好方向，则设置 $\vec{u}_{\text{pref}} = (0,1,0,0)$ ，表示偏向此目标的优化。

(2) 解的方向向量

对于每个解 X_i ，我们计算其在多目标空间内的方向向量 $\vec{u}_i$ 来表示其在各目标维度上的贡献。该方向向量由下式给出：

$$\vec{u}_i = \frac{(f_1(X_i), f_2(X_i), f_3(X_i), f_4(X_i))}{\|\vec{f}(X_i)\|} \quad (59)$$

其中, $\vec{f}(X_i)$ 是解的目标向量的范数, 用于归一化。解的方向向量表示了解的相对优化位置, 有助于在多维空间中识别偏好目标的接近程度。

(3) 偏好角度与惩罚项

计算解与偏好方向的夹角 θ_i , 角度越大表示解偏离偏好方向的程度越大, 具体计算如下:

$$\theta_i = \cos^{-1} \left(\frac{\vec{u}_i \cdot \vec{u}_{\text{pref}}}{\|\vec{u}_i\| \|\vec{u}_{\text{pref}}\|} \right) \quad (60)$$

为避免解过度偏离优先目标, 引入角度惩罚项 P_i , 其公式为:

$$P_i = e^{k \cdot \theta_i} \quad (61)$$

其中, k 为惩罚强度控制参数。角度惩罚项的引入可以减少偏离解的优先级, 集中解向优先目标方向, 有助于在保持多样性的基础上增强解的集中性。

(4) 带偏好的综合评价函数

带偏好的综合评价函数 $F_{\text{pref}}(X_i)$ 将偏好权重和角度惩罚结合, 定义为:

$$F_{\text{pref}}(X_i) = \sum_{m=1}^4 f_m^{\lambda}(X_i) + P_i \quad (62)$$

评价函数越小, 解越符合偏好要求。这种综合评价使得解在偏好方向上的优化效果更强, 避免解在非偏好方向上过度分散。

Step 3 基于偏好评价的选择

具体而言, 基于偏好评价的选择机制通过对非支配解层级进行偏好评价排序, 实现对偏好目标的优先选择。我们根据 $F_{\text{pref}}(X_i)$ 的大小对解进行排序, 优先选择评价值较小的解, 即偏好符合度更高的解。这种排序和选择策略通过优先选择偏好适应性较高的解, 可以更有效地引导后续种群向优先目标聚集, 提升解的整体偏好导向效果。引入决策者偏好评价的改进 NSGA-III 算法的基本步骤如下:

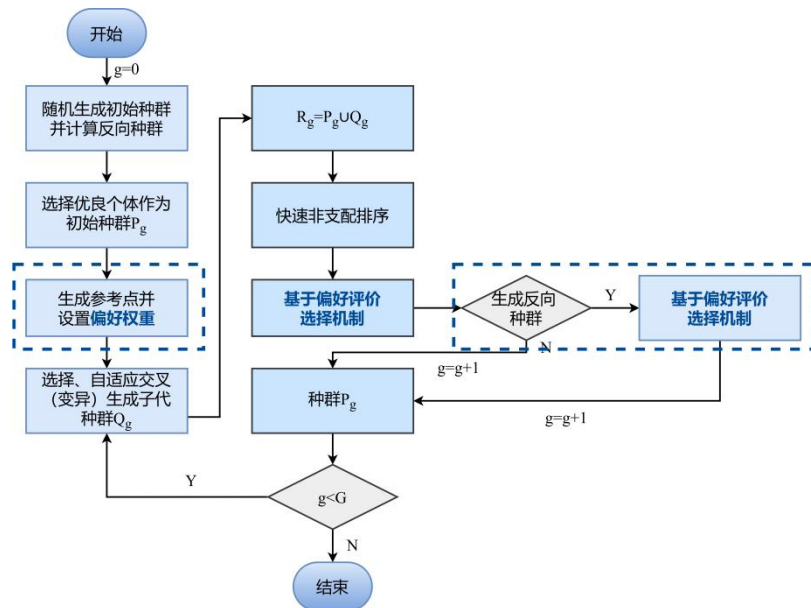


图 18 引入偏好评价选择机制的 NSGA-III 算法流程图

6.4 “一品多仓” 品类分仓规划模型的求解

6.4.1 算法迭代结果分析

本节通过可视化展示了引入偏好评价机制后的算法迭代过程，旨在分析偏好评价机制对各项优化指标的提升效果。引入偏好评价后，算法在优化过程中更聚焦于优先目标品类关联度的提升，同时兼顾其他指标的优化效果，从而实现了多目标优化的均衡推进。通过观察各项指标随迭代次数的变化情况，可以更直观地理解偏好评价机制对算法收敛和目标达成的影响。

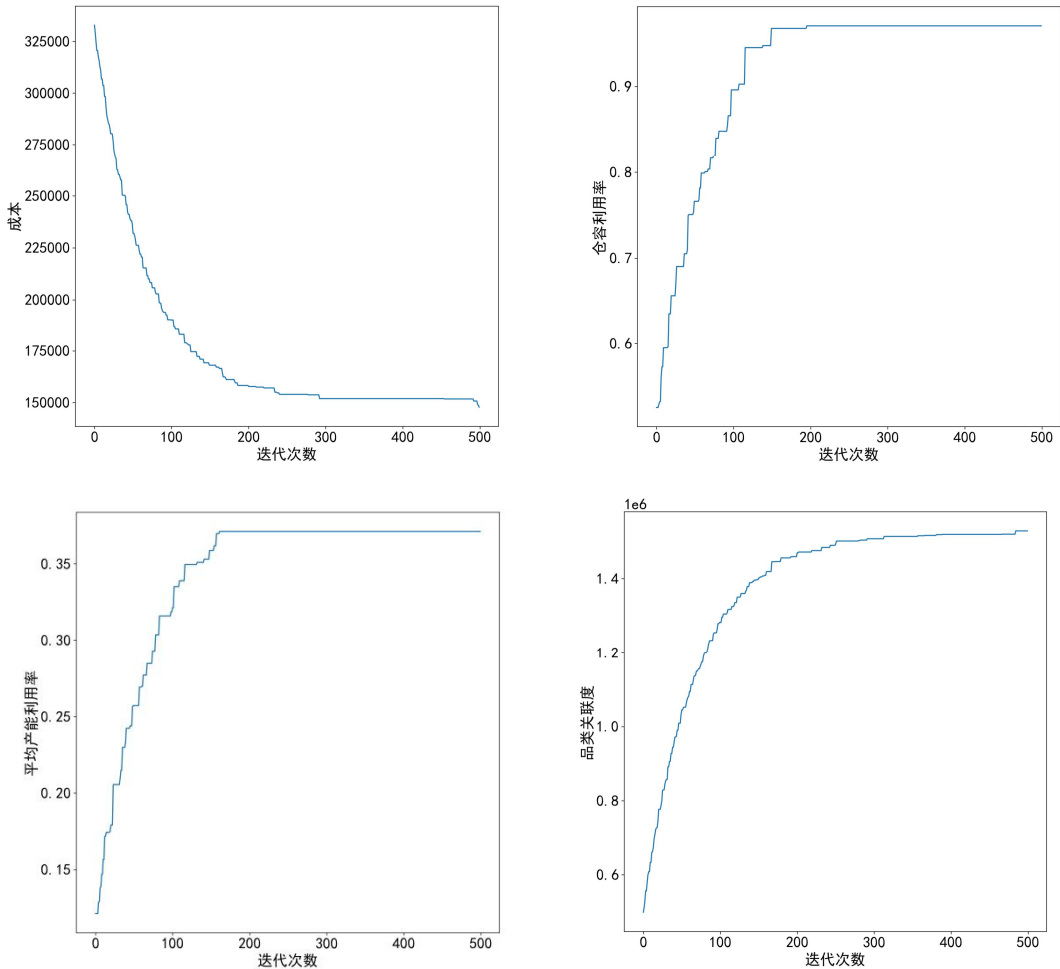


图 19 引入偏好评价后的算法迭代图

从迭代图可以看出，引入偏好评价后，品类关联度这一优先目标迅速提升，并在较少的迭代次数内达到了较高的水平，表现出明显的优化效果。其他指标，如仓库利用率、产能利用率以及成本，也在迭代过程中稳步提升，说明偏好评价机制在突出优先目标的同时，依然兼顾了整体优化，使得算法能够平衡各项仓储资源配置需求。

综上所述，引入偏好评价机制后，算法在品类关联度等优先目标上达成了显著的提升效果，同时实现了其他指标的稳步优化。这一结果展示了偏好评价机制在多目标优化中的有效性，使得算法能够更好地满足决策者的需求，为后续的仓储布局和资源管理提供了重要的数据支持和理论依据。

6.4.2 “一品多仓” 品类分仓结果分析

表 9 “一品多仓” 品类分仓结果

品类	仓库 1	仓库 2	仓库 3
category1	21	87	108
category31	52	96	136
category61	1	118	130
category91	36	105	127
category121	13	105	106
category151	13	130	131
category181	48	60	108
category211	54	105	136
category241	1	60	65
category271	21	60	108
category301	48	60	108
category331	60	108	121

在“一品多仓”分仓结果中，与“一品一仓”分仓策略相比，品类的分配更加灵活和分散。多个品类被分布在不同的仓库中，例如，category1 被分配到仓库 21、87 和 108，而 category121 分布在 13、105 和 106 号仓库，这种策略有效降低了单个仓库的压力，使得仓库间的资源分配更加均衡。相比于“一品一仓”，每个品类的存储不再局限于一个仓库，从而更有利于应对突发需求和仓储资源的不均衡分布。

此外，“一品多仓”策略还为不同品类的跨仓协作提供了可能性，进一步提升了供应链的适应性。例如，多个品类在同一仓库出现的频率提高，像 category181 和 category301 在 60 和 108 号仓库中都有存储，这种分布方式有助于实现仓库间的资源共享和优化管理，提升了出库效率和库存周转率。通过这种多仓分布，不仅满足了多品类的存储需求，还为后续仓储和配送优化提供了更多灵活性。

6.4.3 “一品多仓” 品类分仓业务指标分析

在“一品多仓”策略下，我们对部分仓库的业务指标进行了统计，以进一步分析多仓分配策略在品类关联度、平均产能利用率和仓容利用率方面的表现。表 X 展示了不同仓库在实施“一品多仓”策略后所取得的各项业务指标，为对比“一品一仓”策略的资源利用效率提供了数据基础。

表 10 部分仓库 “一品一仓” 品类分仓业务指标统计

仓库编号	品类关联度	平均产能利用率	仓容利用率
4	87891	0.290744297	0.981785267
13	123417	0.446344671	0.977072258
21	48230	0.46296359	0.996802224
26	77194	0.748548469	0.999724485
60	499463	0.936938326	0.961915628
65	49572	0.725209551	0.972158664
69	29334	0.256521195	0.994096078
105	78145	0.719647024	0.955211395
106	54773	0.4723835	0.909008921

108	67202	0.199340095	0.909450005
121	42723	0.387984251	0.974530308
129	23156	0.199774348	0.870707226
130	59519	0.195140408	0.884248137
132	28029	0.101351649	0.998871593
133	73313	0.301254097	0.971471271

从表中的数据可以看出，与“一品一仓”策略相比，“一品多仓”策略在品类关联度和资源利用率方面表现出了一些差异。仓库编号 60 在“一品多仓”策略下达到了最高的品类关联度（499463），且其产能利用率为 0.937，仓容利用率为 0.962，显示出极高的资源使用效率。这种高关联度的集中存放说明“多仓”策略有助于提高高需求品类的供应能力。此外，仓库编号 26 和 132 的仓容利用率接近 1，分别为 0.999 和 0.999，且产能利用率也保持在较高水平，这表明在“一品多仓”策略下，通过分布存放，不仅能够提升空间利用率，还能满足出库需求。与“一品一仓”相比，“一品多仓”在灵活分仓和均衡资源分配方面展现出明显优势，为不同需求强度的品类提供了更为弹性的存储与管理模式。

6.4.4 同高级品类、同伴型存放结果分析

在本节中，我们对部分仓库在同高级品类、同伴型存放要求下的存放结果进行了统计和分析，以评估不同仓库在满足特定存储条件时的品类和件型占比情况。通过分析这些占比数据，我们可以更清晰地了解各仓库的品类分布和件型配置，为后续的仓储管理优化和资源配置决策提供支持。

表 11 部分仓库品类存放统计

仓库编号	最大高级品类占比	A 件型占比	B 件型占比	C 件型占比
7	45.45	36.36	36.36	27.27
8	54.54	27.27	27.27	45.45
17	33.33	11.11	33.33	55.56
22	27.27	27.27	18.18	54.55
24	31.82	31.82	9.09	59.09
32	20.00	26.67	0.00	73.33
36	33.33	11.11	0.00	88.89
41	33.33	16.67	33.33	50.00
48	73.53	14.71	23.53	61.76
54	60.00	20.00	40.00	40.00
77	45.45	18.18	18.18	63.64
79	50.00	0.00	30.00	70.00
127	50.00	0.00	25.00	75.00
131	40.00	10.00	20.00	70.00
133	43.33	43.33	20.00	36.67

总体来看，各仓库在同高级品类和同伴型的存放方面大体上实现了集中的效果。从表格可以看到，许多仓库的最大高级品类占比都在 30% 以上，部分仓库甚至达到了 50% 以上的占比。这表明在存放策略的优化下，大多数仓库能够有效地集中存放属于同一高级品类的商品，以便更好地进行分类管理。此外，在件型方面，仓库中的 A、B、C 件型商品的占比也体现出了较为明显的集中趋势，有些

仓库的单一件型占比达到或超过 60%，显示出同伴型存放的效果明显。

具体来看，仓库编号 48 和 54 的最大高级品类占比分别达到了 73.53% 和 60.00%，显示了对特定高级品类的高度集中存放倾向。在件型分布上，像编号 32 和 36 的仓库更加倾向于存放 C 件型商品，其占比分别达到了 73.33% 和 88.89%。这种集中存放的策略不仅提升了仓储管理的效率，也为后续的物流和出库操作提供了便利，减少了因多样化存放带来的分拣和管理复杂性。

七、模型的分析与检验

本节对模型在不同情境下的稳健性进行了全面评估。具体而言，通过调整各品类销量和仓库容量两个关键参数，我们考察了模型在成本控制和品类关联度优化方面的表现。实验包含三种情境：销量增加 5%、仓库容量增加 5% 以及第三问的基准情境，对比分析了模型在不同条件下的收敛效果与最终结果。通过这些测试，我们验证了模型在实际应用中应对需求波动和资源调整的适应性与稳健性。

7.1 基于成本的鲁棒性分析

在仓储优化问题中，评估模型的稳健性是关键，尤其是在实际应用中，销量和仓库容量等参数往往会因市场波动或资源调整而发生变化。为了验证本文提出的多目标优化模型在不同情境下的适应能力，本文对各品类销量增加 5%、仓库容量增加 5% 的情境进行了模拟，并与对照情境的成本曲线进行了比较，以观察模型在不同情境中的表现。

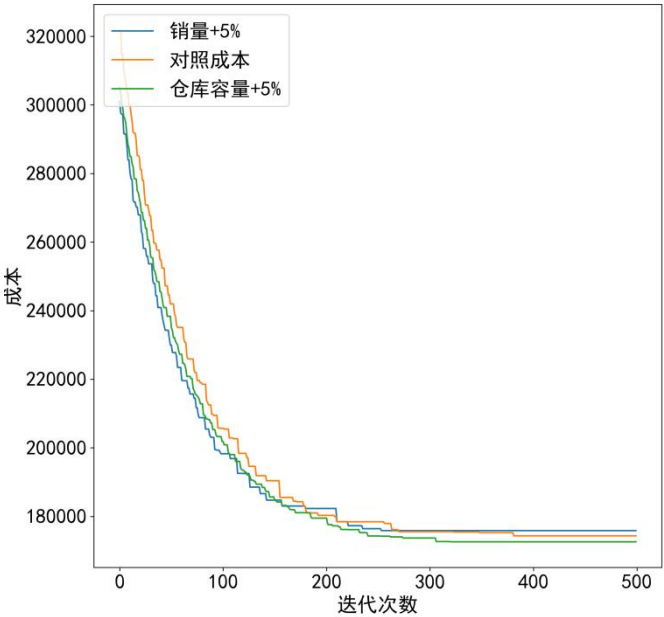


图 20 不同条件下成本的迭代图

从图中可以看出，在销量增加 5% 和仓库容量增加 5% 的情境下，模型的成本变化略有差异。当销量增加 5% 时成本曲线在初期阶段略高于对照情境，并在迭代过程中逐渐收敛至与对照情境相近但略高的水平。这表明，在销量增加的情况下，模型需要应对更高的需求量，因而成本略有上升，但仍能通过优化分配将成本控制在合理范围内。

相反，当仓库容量增加 5% 时，成本曲线在整个迭代过程中略低于对照情境。这表明在仓储容量增加的条件下，模型能够更灵活地配置资源，从而进一步降低成本。无论是在销量增加还是仓库容量增加的情境下，模型最终均能收敛到稳定

的低成本水平，显示出其在不同参数波动下的适应能力和稳健性。

7.2 基于品类关联度的鲁棒性分析

下图展示了在不同情境下基于品类关联度的鲁棒性分析，包括销量增加 5%、对照关联度情境以及仓库容量增加 5%。通过观察不同情境下的品类关联度随迭代次数的变化，可以评估模型在关联度优化方面的适应性与稳健性。

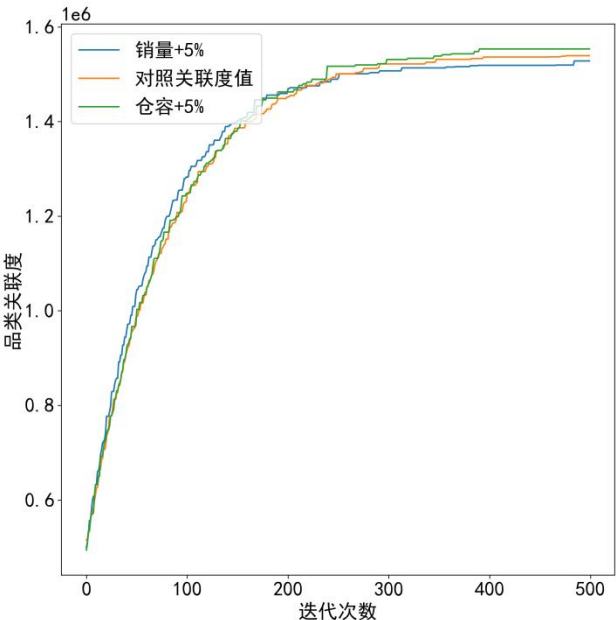


图 21 不同条件下成本的迭代图

从图中可以看出，三种情境下品类关联度都随着迭代次数的增加而逐渐上升，并最终在约 300 次迭代后趋于稳定。当销量增加 5% 时，品类关联度的提升速度较快，初期略高于对照情境（橙线），但最终收敛水平稍低。这表明在销量增加的情况下，模型依然能够找到较优的分仓策略，以尽可能保持高关联度。

另一方面，在仓库容量增加 5% 的情境中，模型在迭代过程中逐渐提升了品类关联度，且最终收敛到略高于对照情境的水平。仓储容量的增加使得模型在资源分配上更加灵活，能够更有效地将关联性高的品类分配到同一仓库。总体来看，模型在不同情境下对品类关联度的优化均表现出较高的适应性，显示出模型在应对需求和容量波动时的稳健性。

综上所述，本文的多目标优化模型在不同情境下对成本和品类关联度均表现出稳健性。在销量增加 5% 时，成本略有上升，但仍能有效控制；在仓库容量增加 5% 时，模型通过更灵活的资源分配实现了成本降低。在品类关联度方面，各情境下的优化效果接近，并在仓库容量增加的情况下略有提升。总体而言，模型在应对需求和容量波动时表现出较强的适应性，能够在控制成本的同时保持较高的品类关联度。

八、模型的评价与推广

本节将对所构建的电商品类货量预测与分仓优化模型进行全面评价，重点评估其在实际应用中的有效性，并探讨模型的优缺点，为进一步优化和改进模型提供依据。

8.1 模型的优点

（1）集成预测方法的高精度：通过集成指数平滑、ARIMA、XGBoost、LightGBM 和 LSTM 模型，充分利用各类算法在不同时间序列特征中的优势，有效提高了货量预测的精确度。多模型融合方法兼顾了短期波动与长期趋势的捕捉，确保了预测结果的可靠性。

（2）多目标优化算法的适应性：针对“一品一仓”和“一品多仓”场景，采用了改进的 NSGA-III 算法，通过引入反向学习和动态交叉变异机制，使算法能够在高维复杂问题中保持良好的收敛性和多样性。该改进使得模型能够在资源配置优化中更加有效地平衡仓储成本和资源利用率。

（3）仓库分配的灵活性：模型在分仓决策时充分考虑了仓库的容量和产能限制，同时引入品类关联度作为分配标准，使得关联度较高的品类集中存放，有效减少了分拣成本和履约时间，提高了仓库运营效率。

8.2 模型的缺点

（1）复杂约束条件的处理难度：模型在分仓优化中引入了多个约束条件（如仓容限制、产能限制、品类关联度约束等），使得求解过程较为复杂，计算时间较长。尽管改进的 NSGA-III 算法在一定程度上提升了求解效率，但面对更大规模的数据集时，计算复杂度仍然较高。

（2）多模型集成带来的参数调优难度：在货量预测过程中，由于使用了多种机器学习和时间序列模型的集成，导致参数设置和模型调优的过程较为繁琐，增加了计算资源的需求。在实际应用中，需要针对不同的品类特点进行细致的参数调节，方能保证预测效果的最佳性。

8.3 模型的推广

本模型结合货量预测与仓储分配的多目标优化方法，可推广应用于其他具备相似业务场景的领域，如零售物流和供应链管理。模型中的集成预测方法和改进的 NSGA-III 算法框架能够适用于具有多种复杂约束和目标的分配优化任务，为各类复杂仓储和资源分配问题提供技术支持与实践参考。

参考文献

- [1] 王雅洁,张成梅,杨鑫,等.基于K-means-LSTM组合算法的地质灾害监测设备故障排查模型设计[J].现代信息科技,2024,8(20):61-67.
- [2] 丁勇,张蓓蓓,吴静.基于配对检验的ARIMA模型在我国甲肝发病数预测中的应用[J].南京医科大学学报(自然科学版),2024,44(10):1456-1461.
- [3] 侯慧,吴文杰,魏瑞增,等.基于注意力机制的CNN-LSTM-XGBoost台风暴雨电力气象混合预测模型[J].智慧电力,2024,52(10):96-102.
- [4] 宋智翔,许伟.基于LightGBM算法的图书馆文献传递服务发展趋势预测[J].信息与管理研究,2024,9(05):55-63.
- [5] 龙弟之,李竞元,李天涯,等.基于改进LSTM的组合式飞轮故障诊断方法[J].航天控制,2024,42(05):83-88.
- [6] 黄锦庭,肖仲华,张立茂.基于BIM和NSGA-III的超高层建筑施工进度多目标优化研究[J].工程管理学报,2024,38(05):111-117.
- [7] 李翟严,胡耀杰,徐礼富,等.基于反向学习的状态空间模型进化算法的充电桩故障诊断预测研究[J].机械设计,2024,41(S1):192-195.
- [8] 王旭健,张峰干,姚敏立.基于动态分解和角度惩罚距离的高维多目标进化算法[J].电子学报,2024,52(08):2773-2785.
- [9] 喻寅昀,李从东,付业林,等.考虑决策者偏好结构的交叉效率评价方法[J].中国管理科学,2024,32(08):117-126.

附录

附录 1	移动平均处理异常值的程序	44
附录 2	随机森林填补缺失值的程序	45
附录 3	指数平滑、ARIMA 预测库存量的程序	47
附录 4	XGBoost、LightGBM、LSTM 预测销售量的程序	50
附录 5	改进的 NSGA-III 算法的程序	57
附录 6	引入偏好评价的改进的 NSGA-III 算法的程序	61

附录 1 移动平均处理异常值的程序

```
import pandas as pd
import numpy as np

# 加载和准备数据
file_path = '附件 2 (23) .csv' # 更新为您的文件路径
data = pd.read_csv(file_path, encoding='ISO-8859-1')
data.columns = ['Category', 'Date', 'Quantity']
data['Date'] = pd.to_datetime(data['Date'], errors='coerce') # 自动检测日期格式
data['Quantity'] = data['Quantity'].astype(float)

# 检查是否有无法解析的日期数据并删除
data = data.dropna(subset=['Date'])
data = data.sort_values(by=['Category', 'Date'])

# 定义异常检测参数
z_threshold = 2.5
window_size = 5 # 用于异常值替换的移动平均窗口大小
common_dates = pd.date_range(start=data['Date'].min(), end=data['Date'].max())

# 创建一个空的 DataFrame 来保存修改后的数据
modified_data_all = pd.DataFrame(columns=['Category', 'Date', 'Quantity'])

# 循环每个类别并处理数据
for category in data['Category'].unique():
    category_data = data[data['Category'] == category].set_index('Date').reindex(common_dates)

    # 计算全局 z-score
    mean_quantity = category_data['Quantity'].mean()
    std_quantity = category_data['Quantity'].std()
    category_data['Z_Score'] = (category_data['Quantity'] - mean_quantity) / std_quantity

    # 找到异常值并使用移动平均值替换
    outliers = category_data[category_data['Z_Score'].abs() > z_threshold]
    modified_data = category_data.copy() # 复制原数据进行替换
    for date in outliers.index:
        # 替换为前后 window_size 的平均值, 跳过 NaN 值
        surrounding_data = category_data.loc[date - pd.DateOffset(days=window_size):date + pd.DateOffset(days=window_size)]
        modified_data['Quantity'].rolling(window=window_size, center=True, min_periods=1).mean()
        modified_data.loc[date, 'Quantity'] = surrounding_data.loc[date]
```

```

# 重置索引并添加类别列，保存到总的 DataFrame
modified_data = modified_data.reset_index().rename(columns={'index': 'Date'})
modified_data['Category'] = category
modified_data_all = pd.concat([modified_data_all, modified_data[['Category',
'Date', 'Quantity']]],
                                ignore_index=True)

# 保存修改后的数据到新文件
modified_data_all.to_csv('附件 2 (23) 异常值处理.csv', index=False,
encoding='utf-8')

```

附录 2 随机森林填补缺失值的程序

```

import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import re
from sklearn.ensemble import RandomForestRegressor
import numpy as np
import os

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 读取缺失信息文件
missing_info = {}
with open('附件二缺失值信息.txt', 'r', encoding='utf-8') as f:
    for line in f:
        match = re.match(r"品类: (.+), 缺失日期: \[(.*?)\]", line)
        if match:
            category = match.group(1)
            dates_str = match.group(2)
            missing_dates = pd.to_datetime(dates_str.split(", "), errors='coerce')
            for date in missing_dates:
                if pd.notna(date):
                    year, month = date.year, date.month
                    if category not in missing_info:
                        missing_info[category] = set()
                    missing_info[category].add((year, month))

# 读取 CSV 文件
data_file = '附件 2(异常值处理完).csv'
data = pd.read_csv(data_file, encoding='gbk')

```

```

data['日期'] = pd.to_datetime(data['日期'], format="%Y/%m/%d", errors='coerce') #
指定日期格式
data['年'] = data['日期'].dt.year
data['月'] = data['日期'].dt.month
data = data.sort_values(by='日期') # 按日期排序

# 输出文件
output_file = '完整数据.csv'

# 先读取输出文件中的现有数据
if os.path.exists(output_file):
    existing_data = pd.read_csv(output_file, encoding='utf-8-sig')
    existing_data['日期'] = pd.to_datetime(existing_data['日期'],
format="%Y/%m/%d", errors='coerce') # 指定日期格式
else:
    existing_data = pd.DataFrame(columns=['品类', '日期', '销量'])

# 遍历每个品类和缺失月份，仅处理有缺失数据的品类
for category, months in missing_info.items():
    processed_years = {'2022': set(), '2023': set()}

    for (year, month) in months:
        # 检查是否已经处理过该年份的三个月范围
        if year in processed_years and processed_years[year].intersection({7, 8, 9}
if year == 2022 else {4, 5, 6}):
            continue

        # 确定缺失月份的具体日期范围
        if year == 2022 and month in [7, 8, 9]:
            months_to_fit = [7, 8, 9]
            full_dates = pd.date_range(start='2022-07-01', end='2022-09-30')
            processed_years['2022'].update(months_to_fit)
        elif year == 2023 and month in [4, 5, 6]:
            months_to_fit = [4, 5, 6]
            full_dates = pd.date_range(start='2023-04-01', end='2023-06-30')
            processed_years['2023'].update(months_to_fit)
        else:
            continue

    # 筛选出当前品类在三个月范围内的所有数据
    plot_data = data[(data['品类'] == category) & (data['年'] == year) & (data['
月'].isin(months_to_fit))]

# 找出缺失的日期：完整日期范围 - 实际已有数据的日期

```

```

existing_dates = set(plot_data['日期'])
missing_dates = [date for date in full_dates if date not in existing_dates]
# print(f' 缺失的日期  ({category}, {year} 年 {months_to_fit} 月 ):
{missing_dates}') # 打印缺失日期以验证

# 使用三个月的数据进行随机森林拟合并预测缺失值
model_data = plot_data.dropna(subset=['销量'])
if model_data.empty:
    continue # 如果没有数据跳过
X_train = np.array((model_data['日期'] - model_data['日期'].min()).dt.days).reshape(-1, 1)
y_train = model_data['销量'].values

# 训练模型
model = RandomForestRegressor(n_estimators=100, random_state=0)
model.fit(X_train, y_train)

# 生成缺失日期的预测
X_missing = np.array((pd.Series(missing_dates) - model_data['日期'].min()).dt.days).reshape(-1, 1)
y_missing_pred = model.predict(X_missing)
y_missing_pred = np.round(y_missing_pred) # 四舍五入取整

# 生成缺失数据的 DataFrame
missing_data = pd.DataFrame({
    '品类': category,
    '日期': pd.Series(missing_dates),
    '销量': y_missing_pred
})

# 检查并去除已存在的数据
for idx, row in missing_data.iterrows():
    # 检查是否已经存在该条记录
    if not ((existing_data['品类'] == row['品类']) & (existing_data['日期'] == row['日期'])).any():
        # 追加单条缺失数据到输出文件
        row.to_frame().T.to_csv(output_file, mode='a', index=False,
header=not os.path.exists(output_file),
encoding='utf-8-sig')
    else:
        print(f'跳过已有数据: 品类 {row['品类']}, 日期 {row['日期']}')

```

附录 3 指数平滑、ARIMA 预测库存量的程序

```
import pandas as pd
```

```

import matplotlib.pyplot as plt
import re
import numpy as np
import os
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.seasonal import seasonal_decompose
from sklearn.metrics import mean_squared_error

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 读取缺失信息文件
missing_info = {}
with open('附件二缺失值信息.txt', 'r', encoding='utf-8') as f:
    for line in f:
        match = re.match(r"品类: (.+), 缺失日期: \[(.*?)\]", line)
        if match:
            category = match.group(1)
            dates_str = match.group(2)
            missing_dates = pd.to_datetime(dates_str.split(", "),
errors='coerce')
            for date in missing_dates:
                if pd.notna(date):
                    year, month = date.year, date.month
                    if category not in missing_info:
                        missing_info[category] = set()
                    missing_info[category].add((year, month))

# 读取 CSV 文件
data_file = '附件 2(异常值处理完).csv'
data = pd.read_csv(data_file, encoding='gbk')
data['日期'] = pd.to_datetime(data['日期'], format="%Y/%m/%d",
errors='coerce')
data['年'] = data['日期'].dt.year
data['月'] = data['日期'].dt.month
data = data.sort_values(by='日期')

# 输出文件
output_file = '完整数据.csv'

# 先读取输出文件中的现有数据
if os.path.exists(output_file):

```

```

existing_data = pd.read_csv(output_file, encoding='utf-8-sig')
existing_data['日期'] = pd.to_datetime(existing_data['日期'],
format="%Y/%m/%d", errors='coerce')
else:
    existing_data = pd.DataFrame(columns=['品类', '日期', '库存量'])

# 遍历每个品类和缺失月份，仅处理有缺失数据的品类
for category, months in missing_info.items():
    processed_years = {'2022': set(), '2023': set()}

    for (year, month) in months:
        if year in processed_years and processed_years[year].intersection({7, 8,
9}) if year == 2022 else {4, 5, 6}):
            continue

        if year == 2022 and month in [7, 8, 9]:
            months_to_fit = [7, 8, 9]
            full_dates = pd.date_range(start='2022-07-01', end='2022-09-30')
            processed_years['2022'].update(months_to_fit)
        elif year == 2023 and month in [4, 5, 6]:
            months_to_fit = [4, 5, 6]
            full_dates = pd.date_range(start='2023-04-01', end='2023-06-30')
            processed_years['2023'].update(months_to_fit)
        else:
            continue

    # 筛选出当前品类在三个月范围内的所有数据
    plot_data = data[(data['品类'] == category) & (data['年'] == year) &
(data['月'].isin(months_to_fit))]

    # 找出缺失的日期
    existing_dates = set(plot_data['日期'])
    missing_dates = [date for date in full_dates if date not in
existing_dates]

    # 使用三个月的数据进行指数平滑和 ARIMA 预测缺失值
    model_data = plot_data.dropna(subset=['库存量'])
    if model_data.empty:
        continue

    y_train = model_data['库存量'].values
    dates_train = model_data['日期']

    # 对数据进行季节分解

```

```

        decomposition = seasonal_decompose(y_train, model='multiplicative',
period=len(y_train) // 2)
        trend = decomposition.trend.dropna()
        seasonal = decomposition.seasonal[:len(y_train)].values
        resid = decomposition.resid.dropna()

        # 使用指数平滑和 ARIMA 对趋势和残差分别进行建模
        # 指数平滑拟合趋势
        trend_model = ExponentialSmoothing(trend, trend='add')
        trend_model_fit = trend_model.fit()
        trend_pred = trend_model_fit.forecast(len(missing_dates))

        # ARIMA 拟合残差
        resid_model = ARIMA(resid, order=(1, 1, 1))
        resid_model_fit = resid_model.fit()
        resid_pred = resid_model_fit.forecast(len(missing_dates))

        # 组合预测结果
        seasonal_cycle = seasonal[:len(missing_dates)]
        y_missing_pred = (trend_pred + seasonal_cycle + resid_pred) / 3    #
组合各个部分的预测结果
        y_missing_pred = np.round(y_missing_pred)

        # 创建缺失数据的 DataFrame
        missing_data = pd.DataFrame({
            '品类': category,
            '日期': pd.Series(missing_dates),
            '销量': y_missing_pred
        })

        # 检查并去除已存在的数据
        for idx, row in missing_data.iterrows():
            if not ((existing_data['品类'] == row['品类']) & (existing_data['日期'] == row['日期'])).any():
                row.to_frame().T.to_csv(output_file, mode='a', index=False,
header=not os.path.exists(output_file),
encoding='utf-8-sig')
            else:
                print(f"跳过已有数据: 品类 {row['品类']}, 日期 {row['日期']}")

```

附录4 XGBoost、LightGBM、LSTM 预测销售量的程序

```

import pandas as pd
import numpy as np

```

```

import matplotlib.pyplot as plt
import tensorflow as tf
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from pmdarima import auto_arima
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import MinMaxScaler

# 设置随机种子 正常预测
np.random.seed(42)
tf.random.set_seed(42)

# 设置中文字体以防止图中出现乱码
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 读取数据文件
data = pd.read_csv('updated 完整数据.csv', encoding='gbk')
data['日期'] = pd.to_datetime(data['日期'])

# 读取不需要预测的品类
with open('全月缺失统计输出.txt', 'r', encoding='gbk', errors='ignore') as f:
    exclude_categories = [line.strip().split()[0] for line in f.readlines()]

# 过滤掉不需要预测的品类
data = data[~data['品类'].isin(exclude_categories)]

# 选择五个品类
categories = data['品类'].unique()[:5] # 假设选择前五个品类
look_back = 30
scaler = MinMaxScaler(feature_range=(0, 1))

# 遍历每个模型和品类进行预测
train_losses = []
val_losses = []

for category in categories:
    # 提取数据
    data_2022 = data[(data['品类'] == category) & (data['日期'] >= '2022-07-01') &
                     (data['日期'] <= '2022-09-30')].copy()

```

```

if len(data_2022) < look_back:
    print(f'{category} 的数据量不足，跳过该品类')
    continue

# 归一化
data_2022['销量'] = scaler.fit_transform(data_2022[['销量']])

# 准备 LSTM 和其他模型的输入
X, y = [], []
for i in range(len(data_2022) - look_back):
    X.append(data_2022['销量'].iloc[i:i + look_back].values.flatten())
    y.append(data_2022['销量'].iloc[i + look_back])
X, y = np.array(X), np.array(y)

# 划分训练集和验证集（80%训练集，20%验证集）
train_size = int(len(X) * 0.8)
X_train, X_val = X[:train_size], X[train_size:]
y_train, y_val = y[:train_size], y[train_size:]

# 模型预测
predictions_train = {}
predictions_val = {}
history_records = {}

# 模型 3: XGBoost
xgb_model = XGBRegressor(n_estimators=100, random_state=42)
xgb_model.fit(X_train, y_train)
predictions_train['XGB'] = xgb_model.predict(X_train)
predictions_val['XGB'] = xgb_model.predict(X_val)
history_records['XGB'] = [np.nan] * 2 # XGBoost 没有直接的损失曲线

# 模型 4: LightGBM
lgb_model = LGBMRegressor(n_estimators=100, random_state=42)
lgb_model.fit(X_train, y_train)
predictions_train['LGB'] = lgb_model.predict(X_train)
predictions_val['LGB'] = lgb_model.predict(X_val)
history_records['LGB'] = [np.nan] * 2 # LightGBM 没有直接的损失曲线

# 模型 5: LSTM
X_train_lstm = X_train.reshape((X_train.shape[0], look_back, 1))
X_val_lstm = X_val.reshape((X_val.shape[0], look_back, 1))
lstm_model = Sequential()

```

```

        lstm_model.add(LSTM(100, input_shape=(X_train_lstm.shape[1],
X_train_lstm.shape[2]), return_sequences=True))
        lstm_model.add(Dropout(0.4))
        lstm_model.add(LSTM(50))
        lstm_model.add(Dense(1))
        lstm_model.compile(loss='mse', optimizer=Adam(learning_rate=0.001))
        history = lstm_model.fit(X_train_lstm, y_train, epochs=50, batch_size=16,
validation_data=(X_val_lstm, y_val), verbose=0)
        predictions_train['LSTM'] = lstm_model.predict(X_train_lstm).flatten()
        predictions_val['LSTM'] = lstm_model.predict(X_val_lstm).flatten()
        history_records['LSTM'] = [history.history['loss'], history.history['val_loss']]

# 反归一化
y_train_inv = scaler.inverse_transform(y_train.reshape(-1, 1)).flatten()
y_val_inv = scaler.inverse_transform(y_val.reshape(-1, 1)).flatten()
for key in predictions_train:
    predictions_train[key] = scaler.inverse_transform(predictions_train[key].reshape(-1, 1)).flatten()
for key in predictions_val:
    predictions_val[key] = scaler.inverse_transform(predictions_val[key].reshape(-1, 1)).flatten()

# 绘制训练集预测图
fig, axs = plt.subplots(1, 3, figsize=(20, 4))
models = ['XGB', 'LGB', 'LSTM']
for i, model in enumerate(models):
    axs[i].plot(y_train_inv, label='真实值', color='blue')
    axs[i].plot(predictions_train[model], label=f'{model} 预测值', linestyle='--')
    axs[i].set_title(f'{category} - {model} 训练集预测')
    axs[i].legend()
plt.show()

# 绘制验证集预测图
fig, axs = plt.subplots(1, 3, figsize=(20, 4))
for i, model in enumerate(models):
    axs[i].plot(y_val_inv, label='真实值', color='blue')
    axs[i].plot(predictions_val[model], label=f'{model} 预测值', linestyle='--')
    axs[i].set_title(f'{category} - {model} 验证集预测')
    axs[i].legend()
plt.show()

# 记录 LSTM 模型的损失曲线
train_losses.append(history_records['LSTM'][0])
val_losses.append(history_records['LSTM'][1])

```

```

# 绘制 LSTM 模型的损失曲线
fig, axs = plt.subplots(1, 5, figsize=(25, 5))
for i, category in enumerate(categories[:len(train_losses)]):
    axs[i].plot(train_losses[i], label='训练集损失', color='blue')
    axs[i].plot(val_losses[i], label='验证集损失', color='orange')
    axs[i].set_title(f'{category} - LSTM 损失曲线')
    axs[i].legend()
plt.show()

```

```

# 特殊预测
import pandas as pd
import numpy as np
import tensorflow as tf
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import MinMaxScaler
import openpyxl

```

```

# 设置随机种子
np.random.seed(42)
tf.random.set_seed(42)

```

```

# 读取数据文件
data = pd.read_csv('updated 完整数据.csv', encoding='gbk')
data['日期'] = pd.to_datetime(data['日期'])

```

```

# 定义特定品类及其训练数据的时间范围
special_cases = {
    'category304': ('2022-07-01', '2022-09-30'),
    'category159': ('2022-04-01', '2022-06-30'),
    'category288': ('2022-07-01', '2022-09-30')
}

```

```

# 设置窗口长度
look_back = 20
scaler = MinMaxScaler(feature_range=(0, 1))

```

```

# 加载 Excel 文件

```

```

excel_path = 'result.xlsx'
wb = openpyxl.load_workbook(excel_path)
sheet = wb['日销量预测结果']

# 逐一处理每个特殊品类
for category, (start_date, end_date) in special_cases.items():
    # 筛选出特定品类和时间范围的数据
    data_train = data[(data['品类'] == category) & (data['日期'] >= start_date) &
                      (data['日期'] <= end_date)].copy()

    # 如果数据量不足，跳过该品类
    if len(data_train) < look_back:
        print(f'{category} 的数据量不足，跳过该品类')
        continue

    # 归一化
    data_train['销量'] = scaler.fit_transform(data_train[['销量']])
    data_train['match'] = data_train['match'].astype(float) # 确保 match 特征为
    浮点数

    # 准备包含工作日/休息日特征的输入数据
    X_train, y_train = [], []
    for i in range(len(data_train) - look_back):
        X_train.append(data_train[['销量', 'match']].iloc[i:i + look_back].values)
        y_train.append(data_train['销量'].iloc[i + look_back])
    X_train, y_train = np.array(X_train), np.array(y_train)

    # 模型 1: XGBoost
    xgb_model = XGBRegressor(n_estimators=100, random_state=42)
    xgb_model.fit(X_train.reshape(X_train.shape[0], -1), y_train)

    # 模型 2: LightGBM
    lgb_model = LGBMRegressor(n_estimators=100, random_state=42)
    lgb_model.fit(X_train.reshape(X_train.shape[0], -1), y_train)

    # 模型 3: LSTM
    X_train_lstm = X_train.reshape((X_train.shape[0], look_back, 2))
    lstm_model = Sequential()
    lstm_model.add(LSTM(100, input_shape=(X_train_lstm.shape[1],
    X_train_lstm.shape[2]), return_sequences=True))
    lstm_model.add(Dropout(0.4))
    lstm_model.add(LSTM(50))
    lstm_model.add(Dense(1))
    lstm_model.compile(loss='mse', optimizer=Adam(learning_rate=0.001))

```

```

lstm_model.fit(X_train_lstm, y_train, epochs=20, batch_size=16, verbose=0)

# 预测未来 90 天
predictions = []
last_sequence = X_train[-1] # 从训练数据的最后一个序列开始预测
for day in range(92): # 预测 90 天
    # 预测
    pred_xgb = xgb_model.predict(last_sequence.reshape(1, -1))[0]
    pred_lgb = lgb_model.predict(last_sequence.reshape(1, -1))[0]
    pred_lstm = lstm_model.predict(last_sequence.reshape(1, look_back,
2)).flatten()[0]

    # 取三者平均作为最终预测
    pred_avg = (pred_xgb + pred_lgb + pred_lstm) / 3

    # 添加小的随机扰动，避免数值趋于平稳
    noise = np.random.normal(0, 0.01) # 调整标准差以控制扰动的大小
    pred_avg += noise

    # 反归一化并四舍五入
    pred_actual = scaler.inverse_transform([[pred_avg]])[0, 0]
    # 如果结果为负数，则设置为 0
    pred_actual = max(0, round(pred_actual))

    predictions.append(pred_actual)

# 更新序列，用于下一天预测
last_sequence = np.roll(last_sequence, -1, axis=0)
# 更新序列时添加 match 特征和新的预测值，使其具有动态变化
last_sequence[-1] = [pred_avg, data_train['match'].iloc[-1]]

# 将预测结果填入 Excel 文件
for row in range(2, sheet.max_row + 1):
    if sheet.cell(row=row, column=1).value == category:
        for col, value in enumerate(predictions, start=2): # 从第 2 列开始填
            sheet.cell(row=row, column=col, value=value)
            print(f"{category} 的预测结果已填入表格")
            break

# 保存 Excel 文件
wb.save(excel_path)
print("预测结果已成功保存至 'result.xlsx'")

```

附录 5 改进的 NSGA-III 算法的程序

```
import numpy as np
import pandas as pd

# 数据读取
# 读取仓库数据
cost_data = pd.read_csv('附件 3.csv', encoding='gb18030')
warehouse_costs = cost_data['仓租日成本'].tolist()
warehouse_capacity = cost_data['仓容上限'].tolist()
warehouse_capacity_per_day = cost_data['产能上限'].tolist()

# 读取 7 月日销售量数据
sales_data = pd.read_excel('result.xlsx', sheet_name='日销量预测结果')
sales_data_july = sales_data.iloc[:, 1:32].values # 7 月的 31 天数据
num_categories = sales_data_july.shape[0] # 商品品类数
num_days = sales_data_july.shape[1] # 天数
num_warehouses = len(warehouse_costs) # 仓库数

# 读取 7 月库存量数据
inventory_data = pd.read_excel('result.xlsx', sheet_name='月库存预测结果')
category_inventory = inventory_data['7 月库存量'].tolist() # 每种品类的 7 月库存量

# 读取关联度数据
association_data = pd.read_csv('附件 4.csv', encoding='gb18030')
nonzero_associations = []
for _, row in association_data.iterrows():
    cat1 = int(row['品类 1'].replace('category', '')) - 1
    cat2 = int(row['品类 2'].replace('category', '')) - 1
    value = row['关联度']
    if value != 0:
        nonzero_associations.append((cat1, cat2, value))

# NSGA-III 参数设置
population_size = 500
max_generations = 2000
mutation_rate = 0.1
crossover_rate = 0.8

# 初始种群生成
def initialize_population():
    population = []
```

```

for _ in range(population_size):
    # 每个品类随机分配到一个仓库
    solution = np.random.randint(0, num_warehouses, size=num_categories)
    population.append(solution)
return np.array(population)

# 适应度函数计算
def evaluate_population(population):
    objectives = []
    for solution in population:
        total_cost, capacity_utilization, demand_utilization, association_score = 0,
0, 0, 0

        # 目标 1: 最小化仓储总成本
        for j in range(num_warehouses):
            if j in solution:
                total_cost += warehouse_costs[j]

        # 目标 2: 最大化仓库容量利用率
        for j in range(num_warehouses):
            warehouse_capacity_used = sum(category_inventory[i] for i in
range(num_categories) if solution[i] == j)
            capacity_utilization += min(warehouse_capacity_used /
warehouse_capacity[j], 1)

        # 目标 3: 最大化产能利用率
        for j in range(num_warehouses):
            daily_demand = sum(
                sales_data_july[i, d] for i in range(num_categories) if solution[i]
== j for d in range(num_days))
            demand_utilization += min(daily_demand /
(warehouse_capacity_per_day[j] * num_days), 1)

        # 目标 4: 最大化品类关联度
        for (cat1, cat2, value) in nonzero_associations:
            if solution[cat1] == solution[cat2]:
                association_score += value

        objectives.append((total_cost, -capacity_utilization, -demand_utilization,
-association_score))

    return np.array(objectives)

```

```

# 非支配排序和拥挤度计算
def non_dominated_sort(objectives):
    num_individuals = objectives.shape[0]
    ranks = np.zeros(num_individuals)
    domination_counts = np.zeros(num_individuals)
    dominated_solutions = [[] for _ in range(num_individuals)]

    for i in range(num_individuals):
        for j in range(num_individuals):
            if all(objectives[i] <= objectives[j]) and any(objectives[i] <
objectives[j]):
                dominated_solutions[i].append(j)
            elif all(objectives[j] <= objectives[i]) and any(objectives[j] <
objectives[i]):
                domination_counts[i] += 1
        if domination_counts[i] == 0:
            ranks[i] = 1

    frontiers = [[]]
    for i in range(num_individuals):
        if ranks[i] == 1:
            frontiers[0].append(i)

    current_rank = 1
    while frontiers[current_rank - 1]:
        next_front = []
        for i in frontiers[current_rank - 1]:
            for j in dominated_solutions[i]:
                domination_counts[j] -= 1
                if domination_counts[j] == 0:
                    ranks[j] = current_rank + 1
                    next_front.append(j)
            frontiers.append(next_front)
        current_rank += 1

    return frontiers[:-1]

```

```

# 交叉操作
def crossover(parent1, parent2):
    if np.random.rand() < crossover_rate:
        crossover_point = np.random.randint(1, num_categories - 1)
        child1 = np.concatenate((parent1[:crossover_point],

```

```

parent2[crossover_point:]))
    child2 = np.concatenate((parent2[:crossover_point],
parent1[crossover_point:]))
    return child1, child2
return parent1, parent2

```

变异操作

```

def mutate(individual):
    if np.random.rand() < mutation_rate:
        mutation_point = np.random.randint(num_categories)
        individual[mutation_point] = np.random.randint(num_warehouses)
    return individual

```

选择机制（基于参考点的选择机制）

```

def select_population(population, objectives, frontiers):
    selected_population = []
    for front in frontiers:
        if len(selected_population) + len(front) <= population_size:
            selected_population.extend(front)
        else:
            selected_population.extend(front[:population_size -
len(selected_population)])
            break
    return population[selected_population]

```

NSGA-III 主函数

```

def nsga_iii():
    population = initialize_population()
    for generation in range(max_generations):
        objectives = evaluate_population(population)
        frontiers = non_dominated_sort(objectives)

        # 父代选择和生成新种群
        new_population = []
        while len(new_population) < population_size:
            parent1, parent2 = population[np.random.choice(len(population), 2,
replace=False)]
            child1, child2 = crossover(parent1, parent2)
            new_population.extend([mutate(child1), mutate(child2)])

```

更新种群

```

        population = select_population(np.array(new_population), objectives,
frontiers)

        # 输出当前代数的最优解
        if generation % 10 == 0:
            print(f'Generation {generation}: Best Solution Objectives:',
objectives[frontiers[0][0]])

        # 返回最终的帕累托前沿解
        objectives = evaluate_population(population)
        frontiers = non_dominated_sort(objectives)
        pareto_front = [population[i] for i in frontiers[0]]
        return pareto_front, objectives[frontiers[0]]

# 运行 NSGA-III 算法
pareto_front, pareto_objectives = nsga_iii()

# 输出最终结果
print("Pareto Front Solutions:")
for solution, obj in zip(pareto_front, pareto_objectives):
    print("Solution:", solution, "Objectives:", obj)

```

附录 6 引入偏好评价的改进的 NSGA-III 算法的程序

```

import numpy as np
import pandas as pd
import random

# 数据读取
# 读取仓库数据
cost_data = pd.read_csv('附件 3.csv', encoding='gb18030')
warehouse_costs = cost_data['仓租日成本'].tolist()
warehouse_capacity = cost_data['仓容上限'].tolist()
warehouse_capacity_per_day = cost_data['产能上限'].tolist()

# 读取 7 月日销售量数据
sales_data = pd.read_excel('result.xlsx', sheet_name='日销量预测结果')
sales_data_july = sales_data.iloc[:, 1:32].values # 7 月的 31 天数据
num_categories = sales_data_july.shape[0] # 商品品类数
num_days = sales_data_july.shape[1] # 天数
num_warehouses = len(warehouse_costs) # 仓库数

# 读取 7 月库存量数据
inventory_data = pd.read_excel('result.xlsx', sheet_name='月库存预测结果')

```

```
category_inventory = inventory_data['7 月库存量'].tolist() # 每种品类的 7 月库存量
```

```
# 读取关联度数据
```

```
association_data = pd.read_csv('附件 4.csv', encoding='gb18030')
```

```
nonzero_associations = []
```

```
for _, row in association_data.iterrows():
```

```
    cat1 = int(row['品类 1'].replace('category', '')) - 1
```

```
    cat2 = int(row['品类 2'].replace('category', '')) - 1
```

```
    value = row['关联度']
```

```
    if value != 0:
```

```
        nonzero_associations.append((cat1, cat2, value))
```

```
# NSGA-III 参数设置
```

```
population_size = 500
```

```
max_generations = 2000
```

```
mutation_rate = 0.2
```

```
crossover_rate = 0.8
```

```
preference_vector = np.array([1, -1, -1, -1]) # 偏好方向，确保总成本最小化，其他目标最大化
```

```
# 初始种群生成
```

```
def initialize_population():
```

```
    population = []
```

```
    for _ in range(population_size):
```

```
        solution = [random.sample(range(num_warehouses), random.randint(1, 3))
```

```
for _ in range(num_categories)]
```

```
    population.append(solution)
```

```
    return population
```

```
# 适应度函数计算
```

```
def evaluate_population(population):
```

```
    objectives = []
```

```
    for solution in population:
```

```
        total_cost, capacity_utilization, demand_utilization, association_score = 0, 0, 0, 0
```

```
        for j in range(num_warehouses):
```

```
            if any(j in solution[i] for i in range(num_categories)):
```

```
                total_cost += warehouse_costs[j]
```

```
        for j in range(num_warehouses):
```

```
            warehouse_capacity_used = sum(
```

```
                category_inventory[i] / len(solution[i]) for i in
```

```

range(num_categories) if j in solution[i])
        capacity_utilization += min(warehouse_capacity_used /
warehouse_capacity[j], 1)

        for j in range(num_warehouses):
            daily_demand = sum(
                sales_data_july[i, d] / len(solution[i]) for i in
range(num_categories) if j in solution[i] for d in
                range(num_days))
            demand_utilization += min(daily_demand /
(warehouse_capacity_per_day[j] * num_days), 1)

        for (cat1, cat2, value) in nonzero_associations:
            if any(w in solution[cat1] for w in solution[cat2]):
                association_score += value

        objectives.append((total_cost, -capacity_utilization, -demand_utilization,
-association_score))

    return np.array(objectives)

# 计算角度惩罚
def calculate_angle_penalty(objective, preference_vector):
    angle = np.arccos(np.dot(objective, preference_vector) /
(np.linalg.norm(objective) * np.linalg.norm(preference_vector)))
    return angle

# 非支配排序和角度惩罚
def non_dominated_sort_with_penalty(objectives):
    num_individuals = objectives.shape[0]
    ranks = np.zeros(num_individuals)
    domination_counts = np.zeros(num_individuals)
    dominated_solutions = [[] for _ in range(num_individuals)]
    penalties = np.array([calculate_angle_penalty(obj, preference_vector) for obj in
objectives])

    for i in range(num_individuals):
        for j in range(num_individuals):
            if all(objectives[i] <= objectives[j]) and any(objectives[i] <
objectives[j]):
                dominated_solutions[i].append(j)
            elif all(objectives[j] <= objectives[i]) and any(objectives[j] <
objectives[i]):
                domination_counts[i] += 1

```

```

        if domination_counts[i] == 0:
            ranks[i] = 1

frontiers = [[]]
for i in range(num_individuals):
    if ranks[i] == 1:
        frontiers[0].append(i)

current_rank = 1
while frontiers[current_rank - 1]:
    next_front = []
    for i in frontiers[current_rank - 1]:
        for j in dominated_solutions[i]:
            domination_counts[j] -= 1
            if domination_counts[j] == 0:
                ranks[j] = current_rank + 1
                next_front.append(j)
    frontiers.append(next_front)
    current_rank += 1

# 在每个前沿上按角度惩罚进行排序
for front in frontiers:
    front.sort(key=lambda idx: penalties[idx])

return frontiers[:-1]

# 交叉和变异
def crossover(parent1, parent2):
    if np.random.rand() < crossover_rate:
        crossover_point = np.random.randint(1, num_categories - 1)
        child1 = parent1[:crossover_point] + parent2[crossover_point:]
        child2 = parent2[:crossover_point] + parent1[crossover_point:]
        return child1, child2
    return parent1, parent2

def mutate(individual):
    if np.random.rand() < mutation_rate:
        mutation_point = np.random.randint(num_categories)
        individual[mutation_point] = random.sample(range(num_warehouses),
random.randint(1, 3))
    return individual

# 选择机制
def select_population(population, objectives, frontiers):

```

```

selected_population = []
for front in frontiers:
    if len(selected_population) + len(front) <= population_size:
        selected_population.extend(front)
    else:
        selected_population.extend(front[:population_size -
len(selected_population)])
        break
return [population[i] for i in selected_population]

# NSGA-III 主函数
def nsga_iii():
    population = initialize_population()
    for generation in range(max_generations):
        objectives = evaluate_population(population)
        frontiers = non_dominated_sort_with_penalty(objectives)

        new_population = []
        while len(new_population) < population_size:
            idx1, idx2 = np.random.choice(len(population), 2, replace=False)
            parent1, parent2 = population[idx1], population[idx2]
            child1, child2 = crossover(parent1, parent2)
            new_population.extend([mutate(child1), mutate(child2)])

        population = select_population(new_population, objectives, frontiers)

        if generation % 10 == 0:
            print(f'Generation {generation}: Best Solution Objectives:',
objectives[frontiers[0][0]])

        objectives = evaluate_population(population)
        frontiers = non_dominated_sort_with_penalty(objectives)
        pareto_front = [population[i] for i in frontiers[0]]
        return pareto_front, objectives[frontiers[0]]

# 运行 NSGA-III 算法
pareto_front, pareto_objectives = nsga_iii()

# 输出最终结果
print("Pareto Front Solutions:")
for solution, obj in zip(pareto_front, pareto_objectives):
    print("Solution:", solution, "Objectives:", obj)

```